Software Testing

Software Engineering Research Centre

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

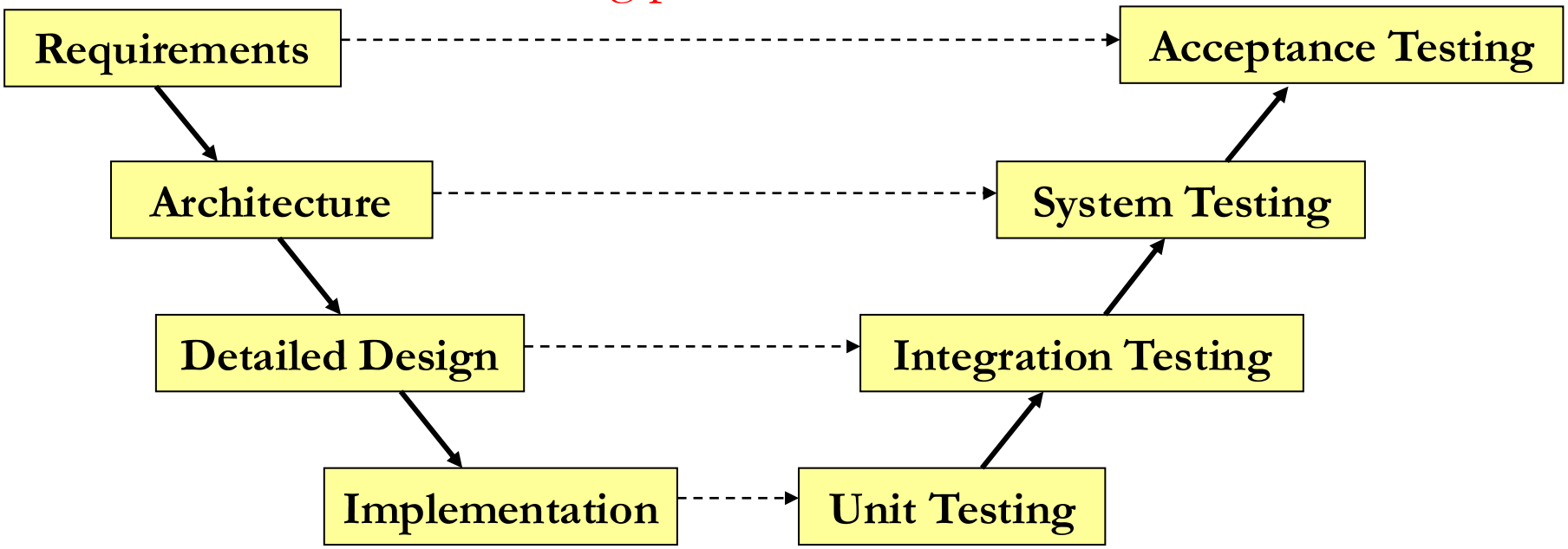
HYDERABAD

TESTING – MODULE 1

Where does testing fit?

SDLC → Requirements → Design → Implementation → **Testing** → Deployment → Maintenance

Testing phases: V model



Testing is not a single phase but spans the **entire lifecycle**

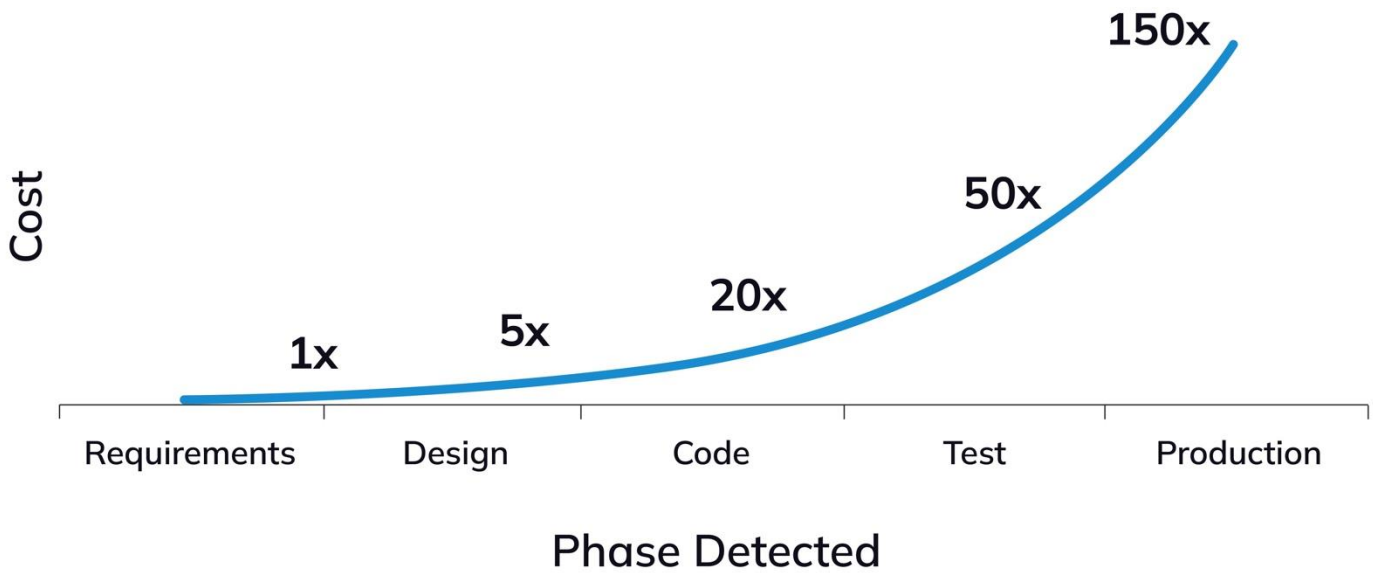
Verification & Validation

- **Verification** — "Are we building the product right?" (Reviews, inspections, static analysis)

- **Validation** — "Are we building the right product?" (execution based testing)

IEEE 1012 (Standard for Software Verification and Validation)

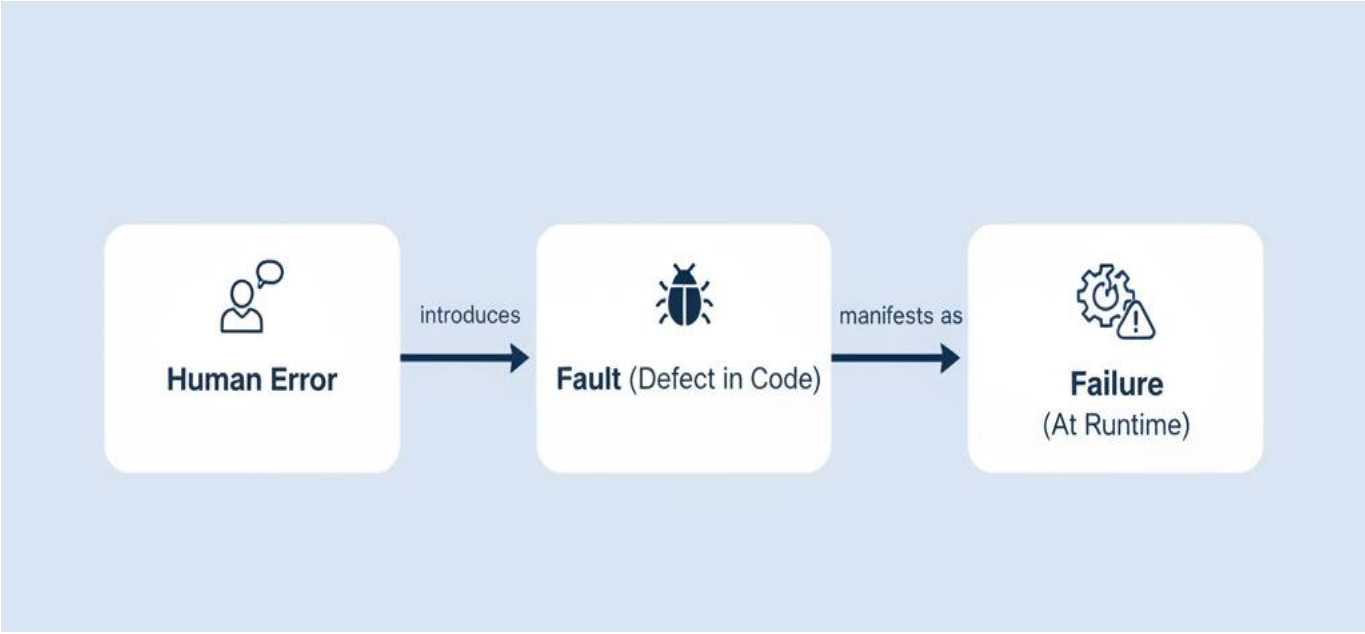
Why Testing Matters



- Defects are inevitable; what matters is *when* they are found.
- Testing provides economic benefit: early detection → lower cost → higher reliability.
- Real-world failures:
  - **Ariane 5 Rocket (1996):** Unhandled overflow → explosion.
  - **Therac-25 (1985):** Missing validation → radiation overdose.
  - **Knight Capital (2012):** Uninitialized variable → $440M loss

Error, Fault, and Failure…

Term	Definition	Example
Error	Human mistake during design or coding	Using wrong variable
Fault (Defect)	Flaw in software code	Off-by-one index
Failure	System deviation during execution	Application crash



Test Case

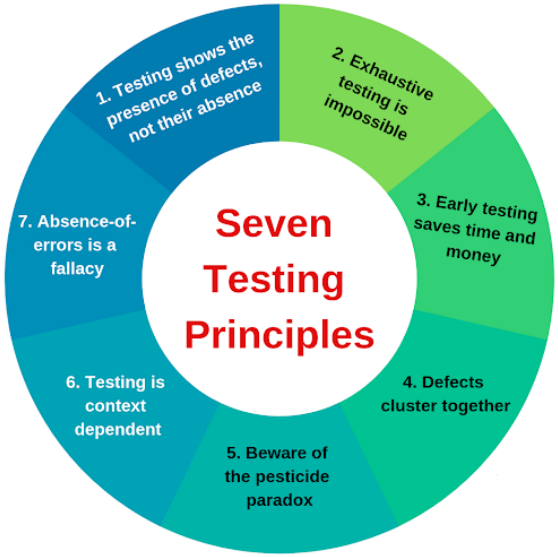
- A triplet of [Input, Steps, Expected behavior]
- (IEEE Standard 829-1983) *Documentation specifying inputs, predicted results, and a set of execution conditions for a test item."*

- *Purpose:* to **get some information about the system behavior**

Goals and philosophies of Testing

Objectives:

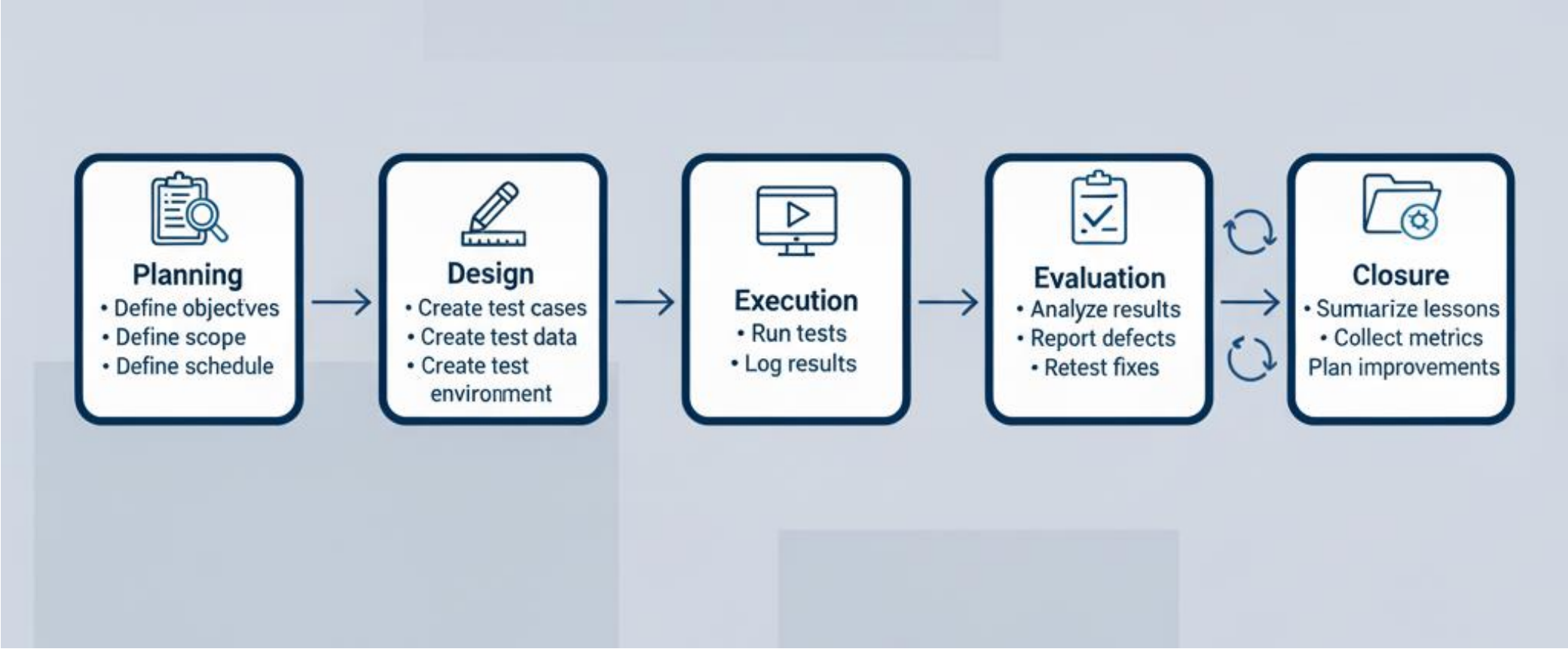
- Detect defects systematically.
- Verify requirement fulfillment.
- Build confidence in product quality.
- Provide information for decision-making.



Principles: (ISTQB "7 Testing Principles")

- Testing shows presence, not absence, of defects.
- Exhaustive testing is impossible — use risk-based selection.
- Early testing saves time and cost.
- Defects cluster in few modules.
- Pesticide paradox: same tests stop finding new bugs — tests must evolve.
- Testing depends on context (safety-critical vs. web app).
- Absence-of-errors fallacy — fixing defects ≠ success if product is unusable.

Structured testing workflow



Next step is to explore how we can verify & validate the artifacts (specifically code) !

Static Vs Dynamic Testing

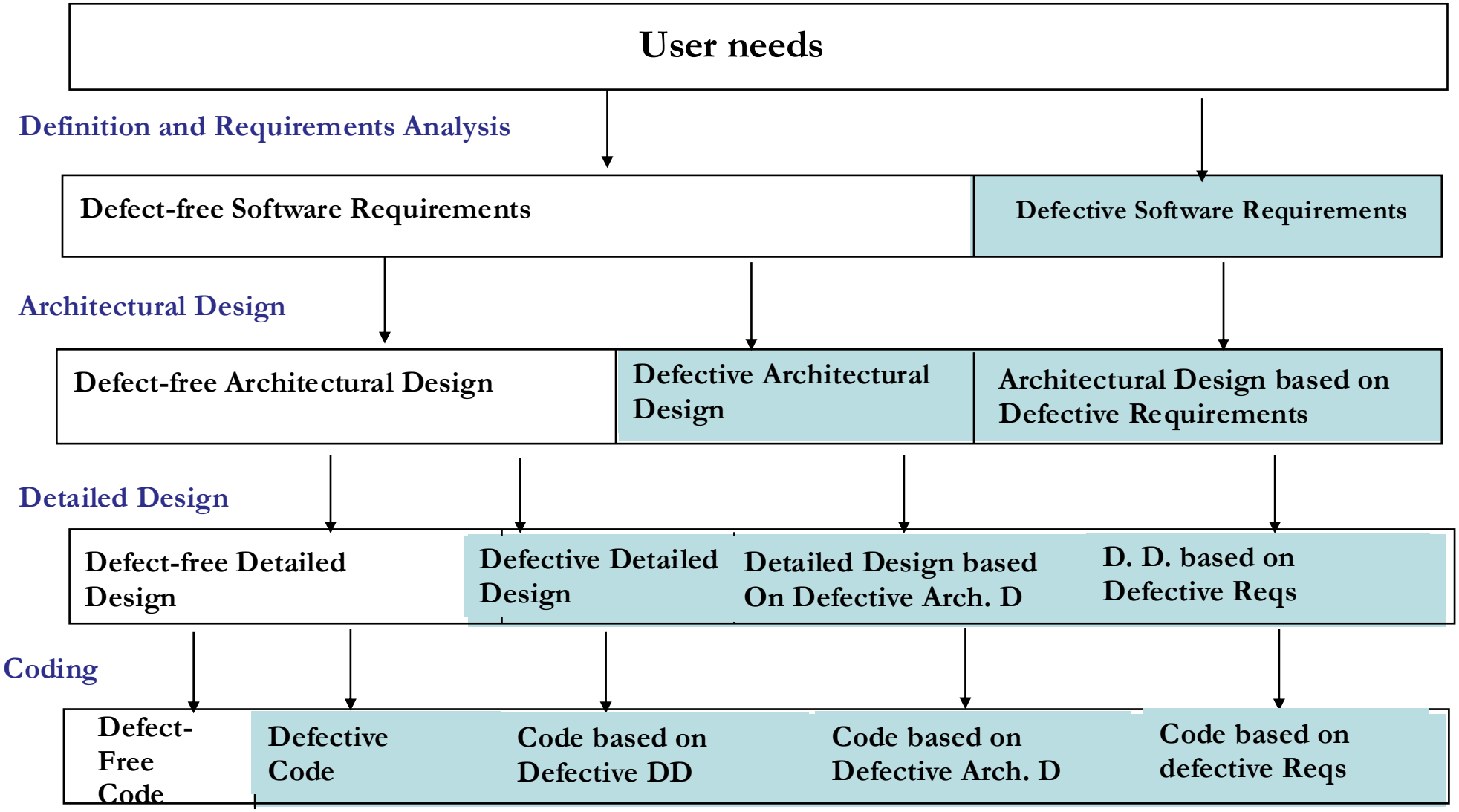
Aspect	Static Testing	Dynamic Testing
Purpose	Prevent defects early	Detect defects after execution
Techniques	Inspections, Walkthroughs, Static analysis	Unit, Integration, System, Acceptance testing
Performed On	Requirements, design docs, code	Executable software
Tools	Code analyzers (SonarQube, PMD, ESLint)	JUnit, Selenium, PyTest
Outcome	Defect prevention	Defect detection

Preventing Defects before they happen…

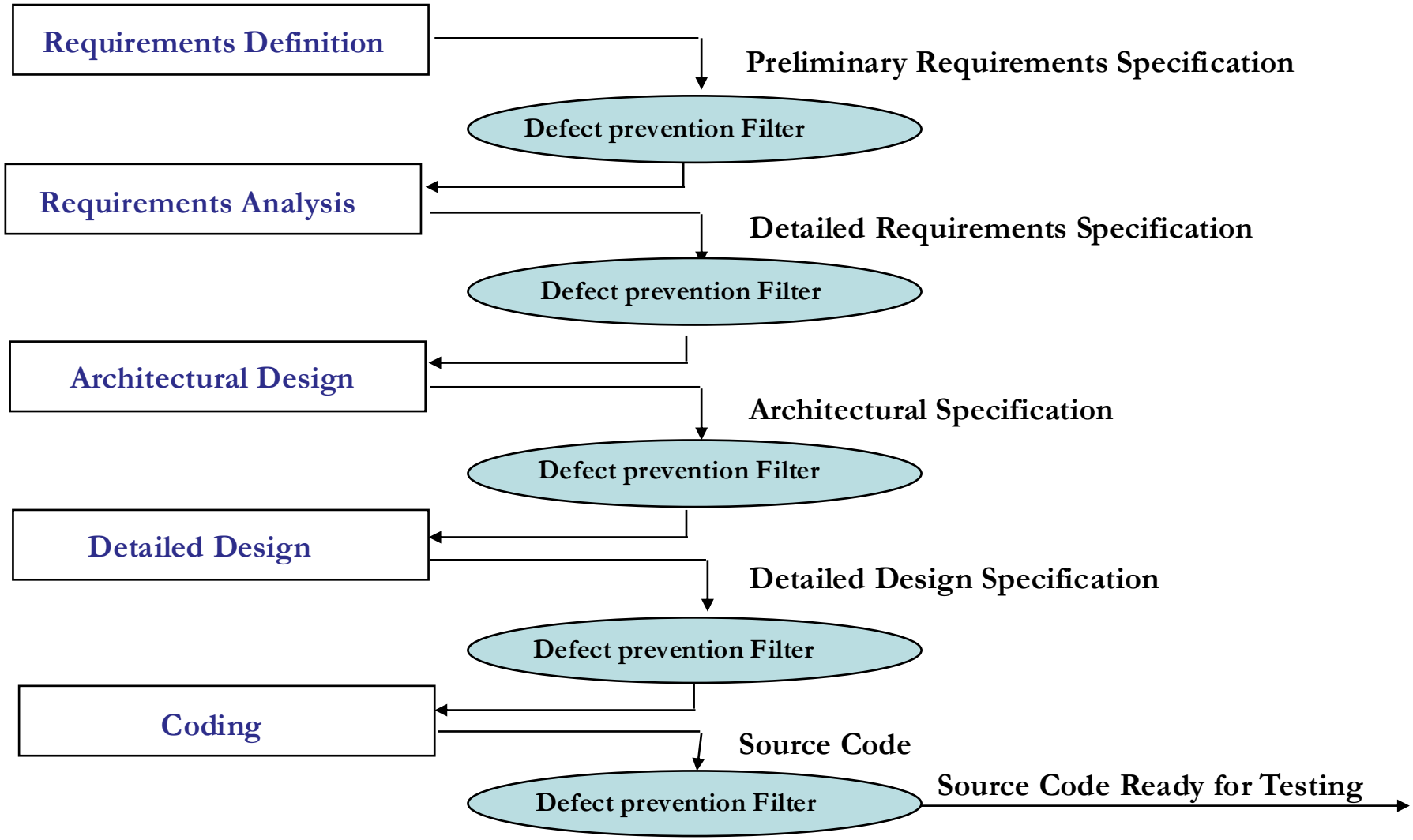
Detecting errors late in the development cycle is expensive

- E.g. A requirements defect that is found only at testing costs almost 100 times more to fix than if it had been found and fixed at requirements itself
- Need to rework not only the requirements doc, but all other deliverables produced from it: change the design, change the code, rerun tests!
- The earlier in the lifecycle we find problems, the cheaper they are to fix

Error Introduction and Propagation



Multiple Stages of Defect Prevention



Inspections & Static Code Analysis

Inspections

- Inspections are formal or informal evaluations of code, aimed at preventing defects and improving code quality through collective review.

Static Code Analysis

- Static code analysis refers to the automated analysis of source code without executing it, focusing on detecting potential errors, code smells, and security vulnerabilities

Both practices enhance code reliability, facilitate adherence to coding standards, and reduce development costs by catching issues early

Inspections often rely on human judgment, enabling team collaboration and knowledge sharing, while **static code analysis** provides objective metrics and identifies issues that might be overlooked.

Current processes used for inspections

- **Code Reviews:**
  - Peer review process where developers examine each other's code.
  - Uses tools like GitHub, Bitbucket.
- **Pair Programming:** Two developers collaborate on a single piece of code, allowing real-time feedback and inspection.
- **Formal Inspections:** A structured evaluation involving multiple stakeholders, including developers, QA, and project managers.
- **Checklists:** Utilizing predefined lists of potential issues or best practices to guide reviews (e.g., consistency, readability).
- **Tools for Static Analysis:** Utilize various tools that automate code inspections
  - SonarQube
  - Checkstyle
  - PMD
  - FindBugs/SpotBugs

Static Program Analysis

- Analyze the source code without executing it to identify potential defects, vulnerabilities and performance issues
- Involves data flow analysis, control flow analysis, and semantic analysis

Example: Using a static analysis tool like pylint to analyze a Python codebase for potential issues such as unused variables, missing docstrings, or code complexity violations.

Static Code Analysis - Examples

```
public void process (String input) {

        if (input.length() > 0) {
                System.out.println (input);
        }
}
```

Possible null pointer exception

Static Code Analysis - Examples

```
import java.util.List; public class ItemProcessor {

        public void processItems(List<Item> items) {

                for (int i = 0; i < items.size(); i++) {
                        Item item = items.get(i);
        System.out.println(item.getName());
                }
        }
}
```

Method uses index-based access, which is less efficient compared to using a for-each loop

Static Code Analysis - Examples

```
public void calculateSum (int a, int b) {
        System.out.println (a + b);
        }

public void calculateTotal (int x, int y) {
        System.out.println (x + y);
}
```

Duplicate logic can be refactored

Static Code Analysis - Examples

```
public List<User> getUsers (String userType) {
        String query = "SELECT * FROM users WHERE type = '" + userType + ”’”;
}
```

Vulnerability Identification. - SQL injection

Static Code Analysis - Examples

```
import java.io.FileReader;

public class FileManager {
        public void readFromFile (String filename) {
        FileReader fr = new FileReader(filename);
        }
}
```

Not closing the FileReader properly can lead to resource leak

Next up – Dynamic Testing !!!

Moving from prevention to detection

Static testing (inspections, reviews) = **defect prevention**.

Dynamic testing (execution-based) = **defect detection**.

Purpose: validate actual system behavior against expected results.

Let's now see how testing is structured into levels that align with different system scopes !!!

Levels of Testing

- **Unit Testing**
  - Developer tests individual modules (functions, methods)
- **Integration testing**
  - Put modules together, try to get them working together
  - Integration testing is complete when the different pieces are able to work together
- **System testing**
  - Testing of entire deliverable against specs
- **Acceptance testing**
  - Testing against user needs, often by the user
  - Confirms readiness for deployment

Unit testing

- During unit testing, modules are tested in isolation:
  - If all modules were to be tested together:
    - it may not be easy to determine which module has the error.
- Unit testing reduces debugging effort several folds.
  - Programmers carry out unit testing immediately after they complete the coding of a module.

Unit Testing involves…

- Writing Test Cases
- Framework Usage
- Isolation of the Unit
- Mocking/Stubbing
- Regression Testing
- Test Coverage

TaskManager - Example

```java
import java.util.ArrayList;
import java.util.List;

public class TaskManager {
    private List<String> tasks;

public TaskManager() {
        this.tasks = new ArrayList<>();
    }
    public void addTask(String task) {
        tasks.add(task);
    }
    public void markTaskAsCompleted(String task) {
        tasks.remove(task);
    }
    public List<String> getTasks() {
        return tasks;
    }
}
```

```java
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;
import java.util.List;

public class TaskManagerTest {
    private TaskManager taskManager;

    @Before
    public void setUp() {
        taskManager = new TaskManager();
        taskManager.addTask("Task 1");
        taskManager.addTask("Task 2");
        taskManager.addTask("Task 3");
    }

    @Test
    public void testAddTask() {
        taskManager.addTask("New Task");
        List<String> tasks = taskManager.getTasks();
        assertEquals(4, tasks.size());
        assertTrue(tasks.contains("New Task"));
    }

    @Test
    public void testMarkTaskAsCompleted() {
        taskManager.markTaskAsCompleted("Task 2");
        List<String> tasks = taskManager.getTasks();
        assertEquals(2, tasks.size());
        assertFalse(tasks.contains("Task 2"));
    }

    @Test
    public void testGetTasks() {
        List<String> tasks = taskManager.getTasks();
        assertEquals(3, tasks.size());
        assertTrue(tasks.contains("Task 1"));
        assertTrue(tasks.contains("Task 2"));
        assertTrue(tasks.contains("Task 3"));
    }
}
```

Writing effective unit tests…

1. Understanding Requirements
2. Complex Logic
3. Dependencies (Mocking/Stubbing)
4. Performance Overhead
5. False Positives/Negatives
6. Test Coverage
7. Testing Edge Cases
8. Maintaining Test Independence
9. …

Mocking/Stubbing

```
public class PaymentProcessor {
    private PaymentGateway paymentGateway;

    public PaymentProcessor(PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }

    public boolean processPayment(double amount) {
        // Call the payment gateway to process the payment
        return paymentGateway.charge(amount);
    }
}
```

```
public interface PaymentGateway {
    boolean charge(double amount);
}
```

```
import static org.junit.Assert.*;
import static org.mockito.Mockito.*;

import org.junit.Test;

public class PaymentProcessorTest {

    @Test
    public void testProcessPayment() {
        // Create a mock PaymentGateway
        PaymentGateway mockGateway = mock(PaymentGateway.class);

        // Configure the mock to return true when charge method is called with any
double value
        when(mockGateway.charge(anyDouble())).thenReturn(true);

        // Create PaymentProcessor with the mock PaymentGateway
        PaymentProcessor paymentProcessor = new
PaymentProcessor(mockGateway);

        // Call the method under test
        boolean result = paymentProcessor.processPayment(100.0);

        // Verify that the payment was processed successfully
        assertTrue(result);

        // Verify that the charge method of the mock was called exactly once with
the specified amount
        verify(mockGateway, times(1)).charge(100.0);
    }
}
```

False Positives

```
public class EmailValidator {
    public boolean isValidEmail(String email) {
        // Simplified email validation logic
        return email != null && email.contains("@") && email.contains(".");
    }
}
```

```
import static org.junit.Assert.*;
import org.junit.Test;

public class EmailValidatorTest {

    @Test
    public void testIsValidEmail() {
        EmailValidator emailValidator = new EmailValidator();
        assertTrue(emailValidator.isValidEmail("test@@example.com")); // Incorrect assertion
    }
}
```

Maintaining Test independence

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {

    @Test
    public void testAddition() {
        Calculator calculator = new Calculator();
        assertEquals(5, calculator.add(2, 3));
    }

    @Test
    public void testSubtraction() {
        Calculator calculator = new Calculator();
        assertEquals(2, calculator.subtract(5, 3));
    }

    @Test
    public void testMultiplication() {
        Calculator calculator = new Calculator();
        assertEquals(15, calculator.multiply(5, 3));
    }

    @Test
    public void testDivision() {
        Calculator calculator = new Calculator();
        assertEquals(2, calculator.divide(6, 3));
    }

    @Test(expected = IllegalArgumentException.class)
    public void testDivisionByZero() {
        Calculator calculator = new Calculator();
        calculator.divide(6, 0);
    }
}
```

Integration testing

- After different modules of a system have been coded and unit tested:
  - modules are integrated in steps according to an integration plan
  - partially integrated system is tested at each integration step.

Objectives:

- Gain confidence in the integrity of overall system design
- Ensure proper interaction of components

Simple example…

- Let's consider a scenario where a software application is being developed, consisting of three separate modules: User Interface (UI), Database (DB), and Payment Gateway (PG).

- UI: Handles user interactions and displays information.
- DB: Manages data storage and retrieval.
- PG: Processes payment transactions.

Each module is first tested individually (Unit Testing) to ensure it works correctly on its own.

- UI Test: Checking if the UI displays data correctly.
- DB Test: Ensuring data is stored and retrieved correctly.
- PG Test: Verifying that transactions are processed accurately.

Simple example… integration testing

After unit testing, the modules are combined and tested together.

UI + DB Integration: Checking if the UI correctly displays data fetched from the DB.

DB + PG Integration: Ensuring the DB correctly processes data received from and sent to the PG.

UI + PG Integration: Verifying that the UI can initiate and display results of payment transactions through the PG.

Scenario for Integration Testing:

- A user performs an action on the UI, like placing an order.
- The order data is sent from the UI to the DB for storage.
- The payment details are sent from the UI to the PG for transaction processing.
- The PG sends transaction status back to the UI.
- The UI displays the order and payment status to the user.

Integration Testing Strategies

- Big-bang
- Top-down
- Bottom-up
- Function-at-a-time
- Sandwich

Big Bang Integration Testing

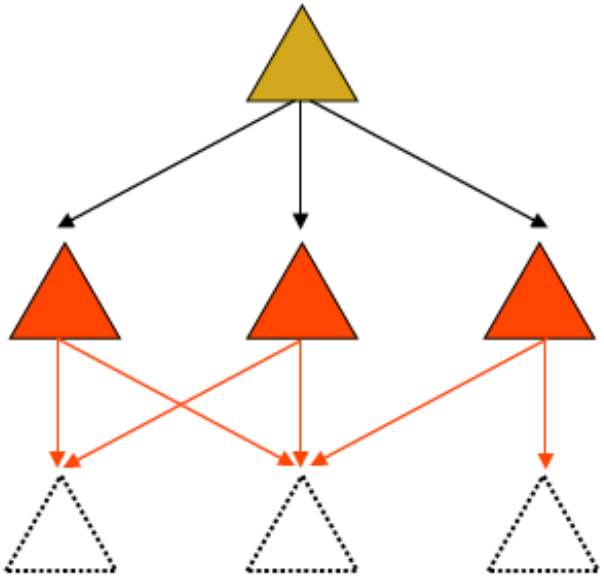
- Big bang approach is the simplest integration testing approach:
  - all the modules are simply put together and tested.
  - this technique is used only for very small systems.

Issues:

- avoids cost of scaffolding (stubs or drivers)
- does not provide any locality for finding faults

Top-down integration testing

- Start with top-level modules
- Use stubs for lower-level modules
- As each level is completed, replace stubs with next level of modules



Pros:

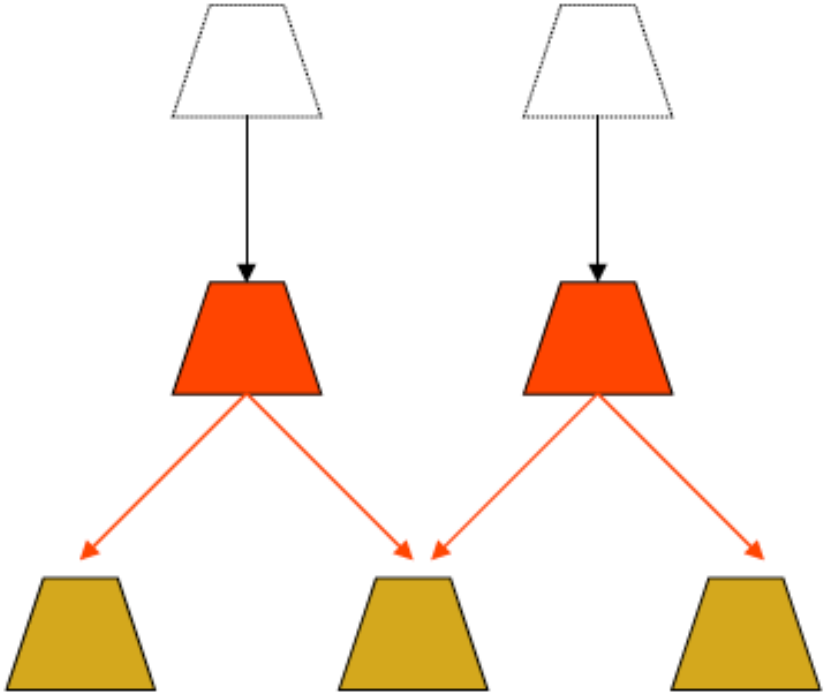
- Always have a top-level system
- Stubs can be written from interface specifications

Cons:

- May delay performance problems until too late
- Stubs can be expensive

Bottom-up Integration Testing

- Start with bottom-level modules
- Use drivers for upper-level modules
- As each level is completed, replace drivers with next level of modules



Pros:

- Primitive functions get most testing
- Drivers are usually cheap

Cons:

- Only have a complete system at the end

Function-at-a-time Integration

- Integrate all modules needed to perform a particular function
- For each function, add another set of modules

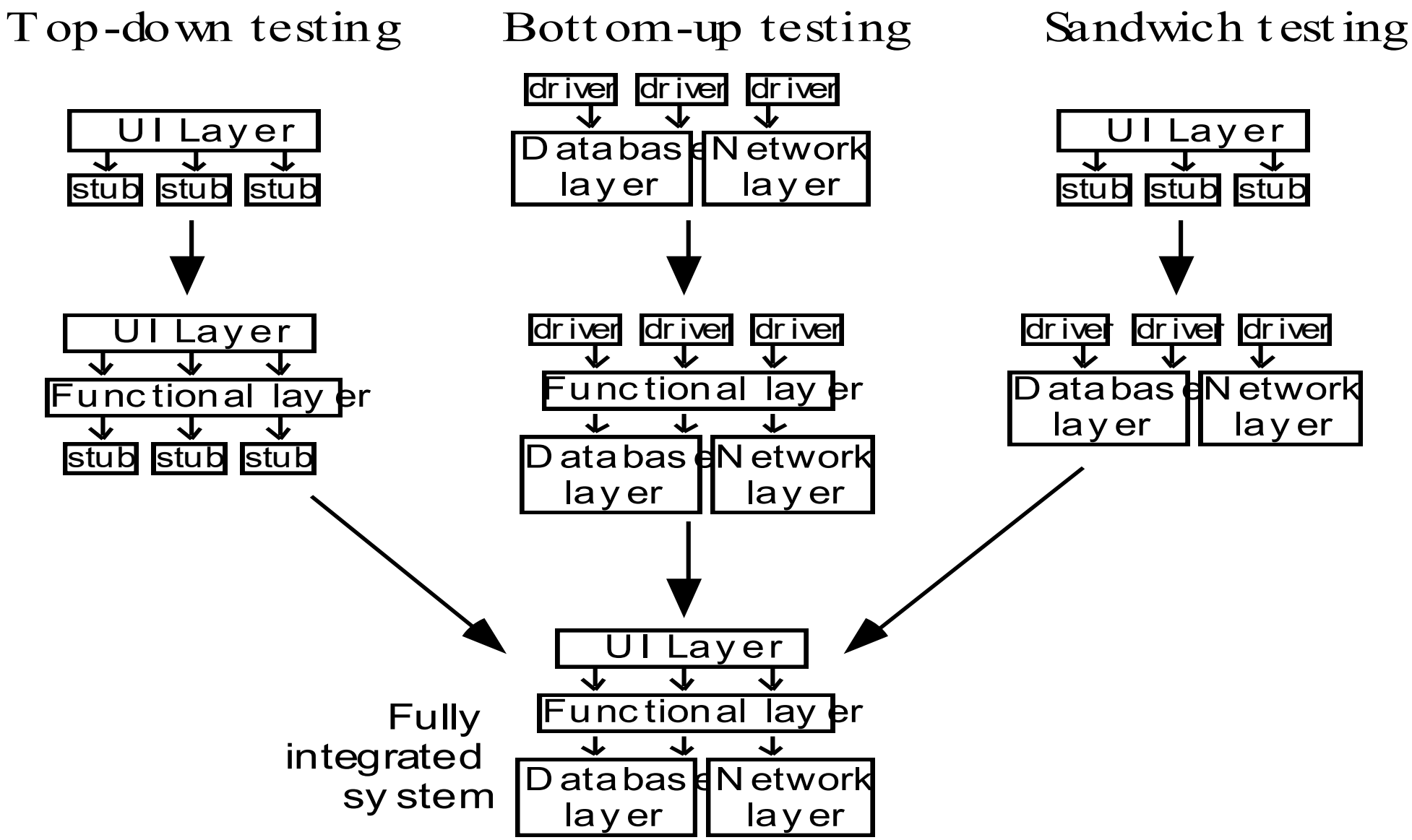
Issues

- Makes for easier test generation
- May postpone function interaction for too long
  - Dependencies may create a problem

Sandwich Integration Testing

- Mixed (or sandwiched) integration testing:
  - uses both top-down and bottom-up testing approaches.
  - Most common approach

Example of different integration strategies



System Testing

Gain confidence in the integrity of the system as a whole

- Ensure compliance with functional requirements
- Ensure compliance with non-functional requirements (for example, performance requirements)

Acceptance Testing

- Testing performed by the customer or end-user himself:
  - to determine whether the system should be accepted or rejected.

Other paths to acceptance:

- Beta testing
  - Distribute system to volunteers
  - Collect change requests, fix, redistribute
  - Collect statistics on beta use
- Shadowing
  - Collect or redistribute real-time use of existing system
  - Compare results
  - Collect statistics

Next up…

Functional & Non-functional Testing

What does the system do **Vs** How well it does it ?

Functional Testing: What the system does (features, behavior).

Example: Does login validate correct credentials?

Test basis: Requirements, use cases, specifications

Techniques: Blackbox, Scenario-based

Non-Functional Testing: How well the system performs under conditions

Example: How fast is the response for a request?

Test basis: Quality attributes (performance, usability, etc.)

Techniques: Load, Stress, Penetration, Usability testing

Functional Testing Techniques

Black-box testing methods:

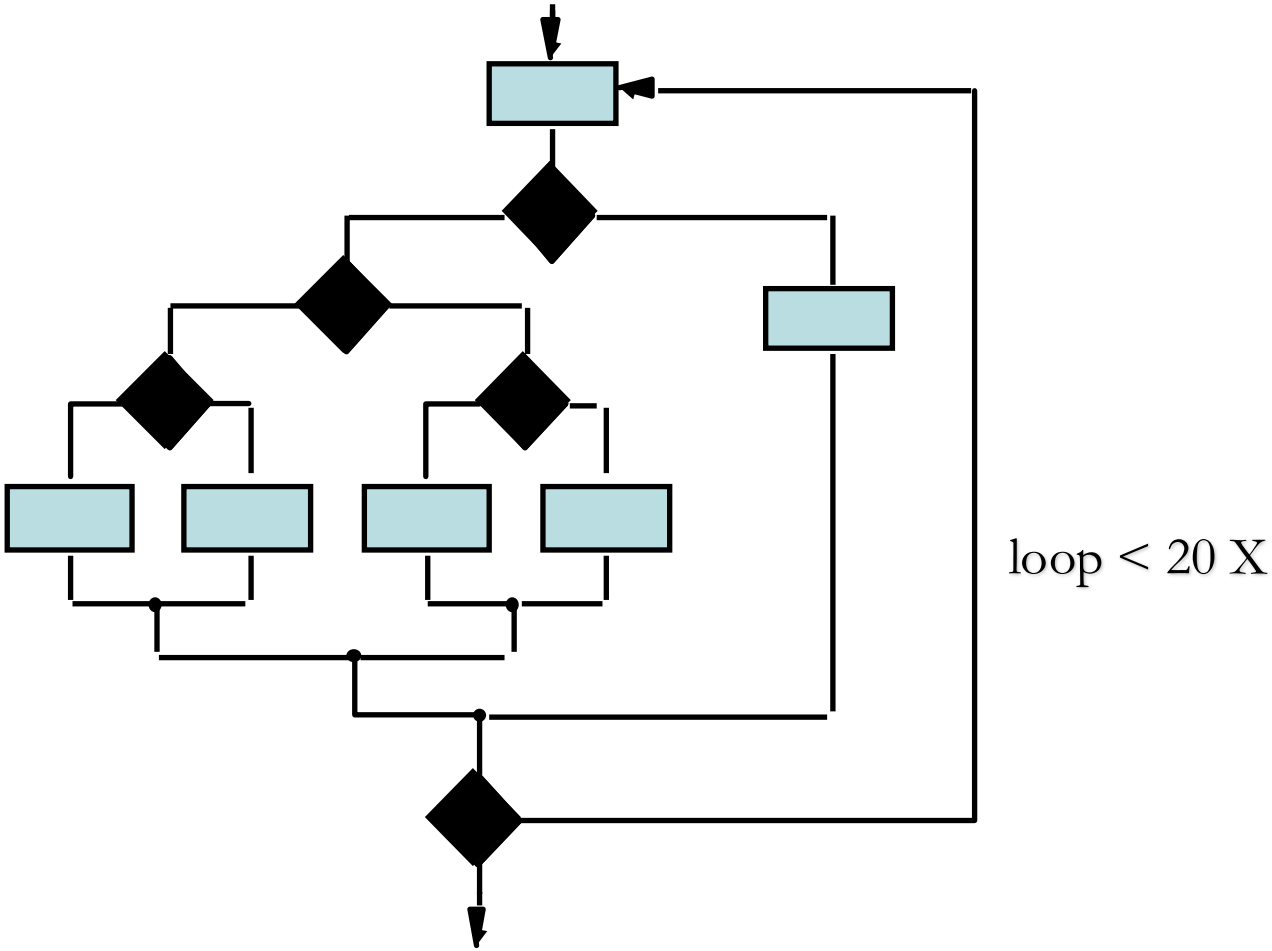
- **Equivalence Partitioning:** Divide input domain into valid/invalid classes
- **Boundary Value Analysis:** Focus on edge values
- **Decision Table Testing:** Handle complex business rules
- **State Transition Testing:** Validate behavior across states

Scenarios and Use Cases: Derive functional tests from UML diagrams or requirement scenarios.

Regression Functional Testing: Ensure existing functionality remains intact after changes.

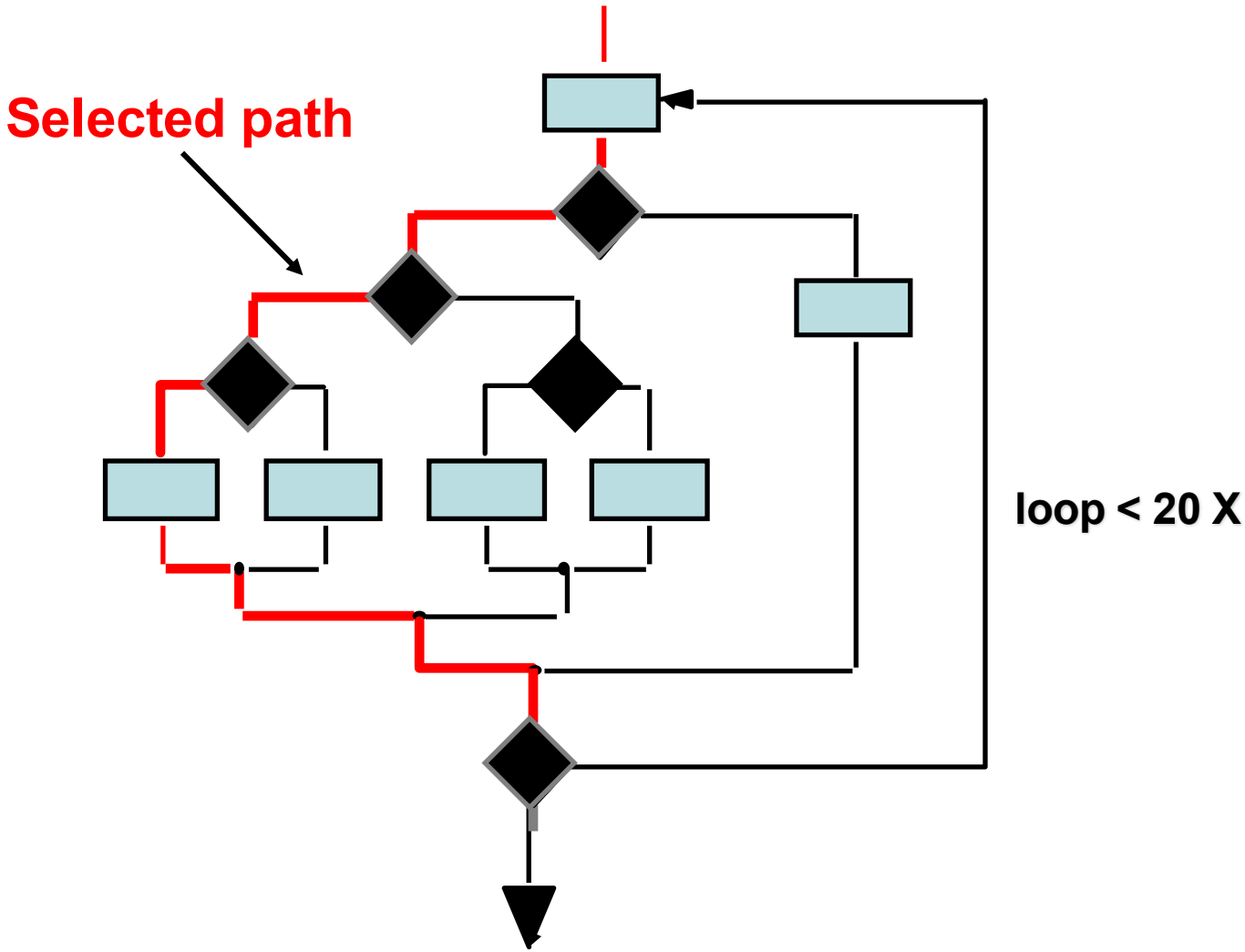
Equivalence Partitioning

Exhaustive Testing



There are 10 possible paths! If we execute one test per millisecond, it would take 3,170 years to test this program!!

Selective Testing



Coverage

- Function coverage: Each function/method executed by at least one test case
- Statement coverage: Each line of code covered by at least one test case (need more test cases than above)
- Path coverage: Every possible path through code covered by at least one test case (need lots of test cases)

Equivalence classes

- It is inappropriate to test by *brute force*, using *every possible* input value
  - Takes a huge amount of time
  - Is impractical
  - Is pointless!

- You should divide the possible inputs into groups which you believe will be treated similarly by all algorithms.
  - Such groups are called *equivalence classes*.
  - A tester needs only to run one test per equivalence class
  - The tester has to
    - understand the required input,
    - appreciate how the software may have been designed

Example of equivalence classes

Valid input is a month number (1-12)

- Equivalence classes are: [-∞..0], [1..12], [13.. ∞]

Combinations of equivalence classes

- Combinatorial explosion means that you cannot realistically test every possible system-wide equivalence class.
  - If there are 4 inputs with 5 possible values there are 5^4 (i.e.625) possible system-wide equivalence classes.
- You should first make sure that at least one test is run with every equivalence class of every individual input.
- You should also test all combinations where an input is likely to *affect the interpretation* of another.
- You should test a few other random combinations of equivalence classes.

Example equivalence class combinations

- One valid input is either 'Metric' or 'US/Imperial'
  - Equivalence classes are:
    - Metric, US/Imperial, Other

- Another valid input is maximum speed: 1 to 750 km/h or 1 to 500 mph
  - Validity depends on whether metric or US/imperial
  - Equivalence classes are:
    - [-∞..0], [1..500], [501..750], [751.. ∞]

- Some test combinations
    - Metric, [1..500] valid
    - US/Imperial, [501..750] invalid
    - Metric, [501..750] valid
    - Metric, [501..750] valid

Boundary Value Analysis

Boundary Value Testing

- More errors in software occur at the boundaries of equivalence classes
- The idea of equivalence class testing should be expanded to specifically test values at the extremes of each equivalence class
  - E.g. The number 0 often causes problems

- *E.g.*: If the valid input is a month number (1-12)
  - Test equivalence classes as before
  - Test 0, 1, 12 and 13 as well as very large positive and negative values

Example Boundaries

Objective: To test whether a given three values can be sides of a triangle

1, 1, 1 (minimum valid values)

1, 0, 1 (invalid: too small)

20, 30, 49 (valid)

20, 30, 50 (invalid: one side equals sum of other two sides)

20, 30, 51 (invalid: one side too long)

Decision Table Testing

Decision tables

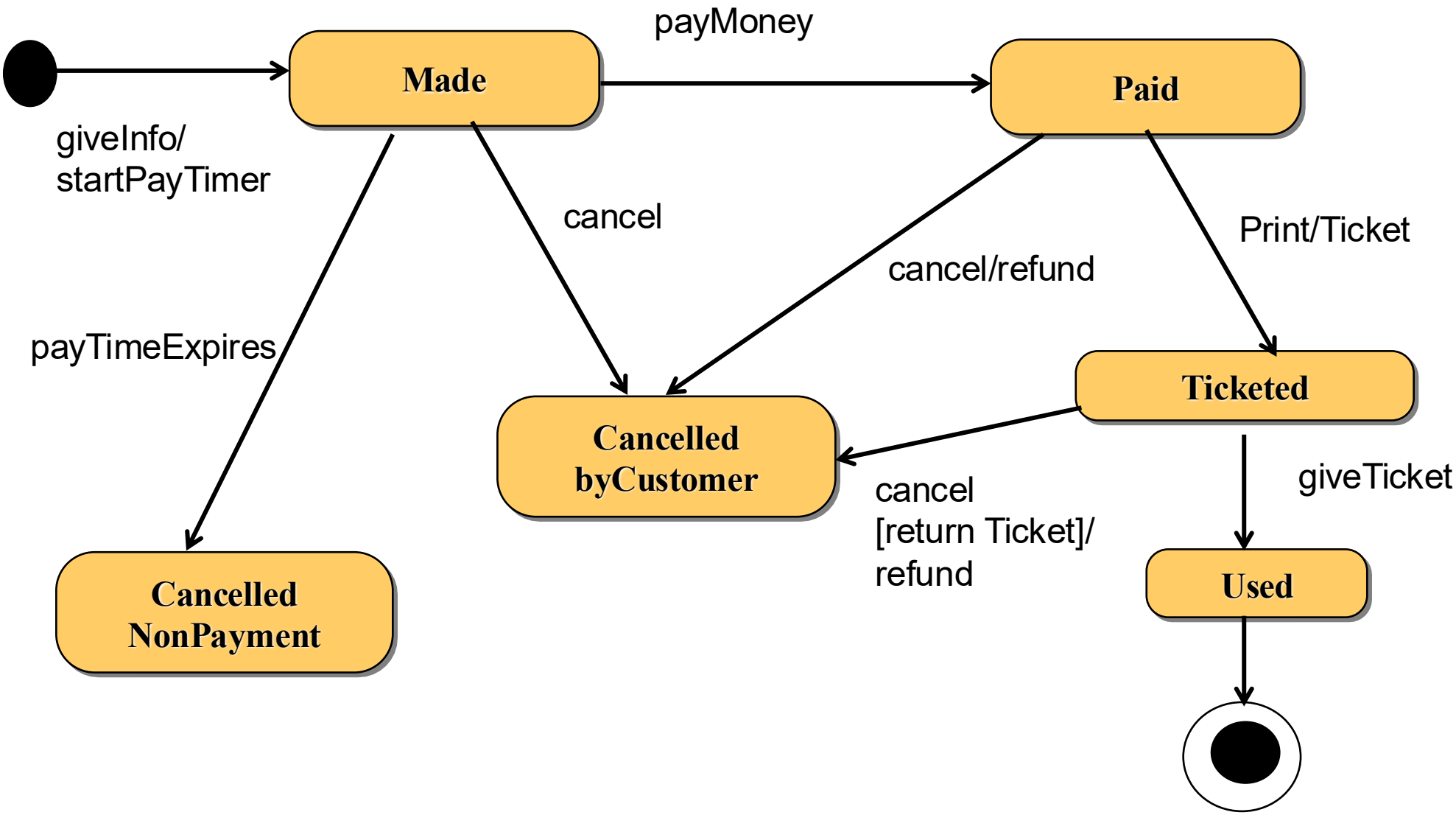
	Rule 1	Rule 2	…	Rule Z
CONDITIONS				
Condition -1				
Condition - 2				
…				
Condition-m				
ACTIONS				
Action - 1				
Action - 2				
…				
Action-n				

Decision table – single decision example

	Rule 1	Rule 2	Rule 3	Rule 4
CONDITIONS				
Married ?	YES	YES	NO	NO
Good Student ?	YES	NO	YES	NO
ACTIONS				
Discount ($)	60	25	50	0

State Transition Testing

Reservation system example



Creating Test Cases

Create a state-transition table

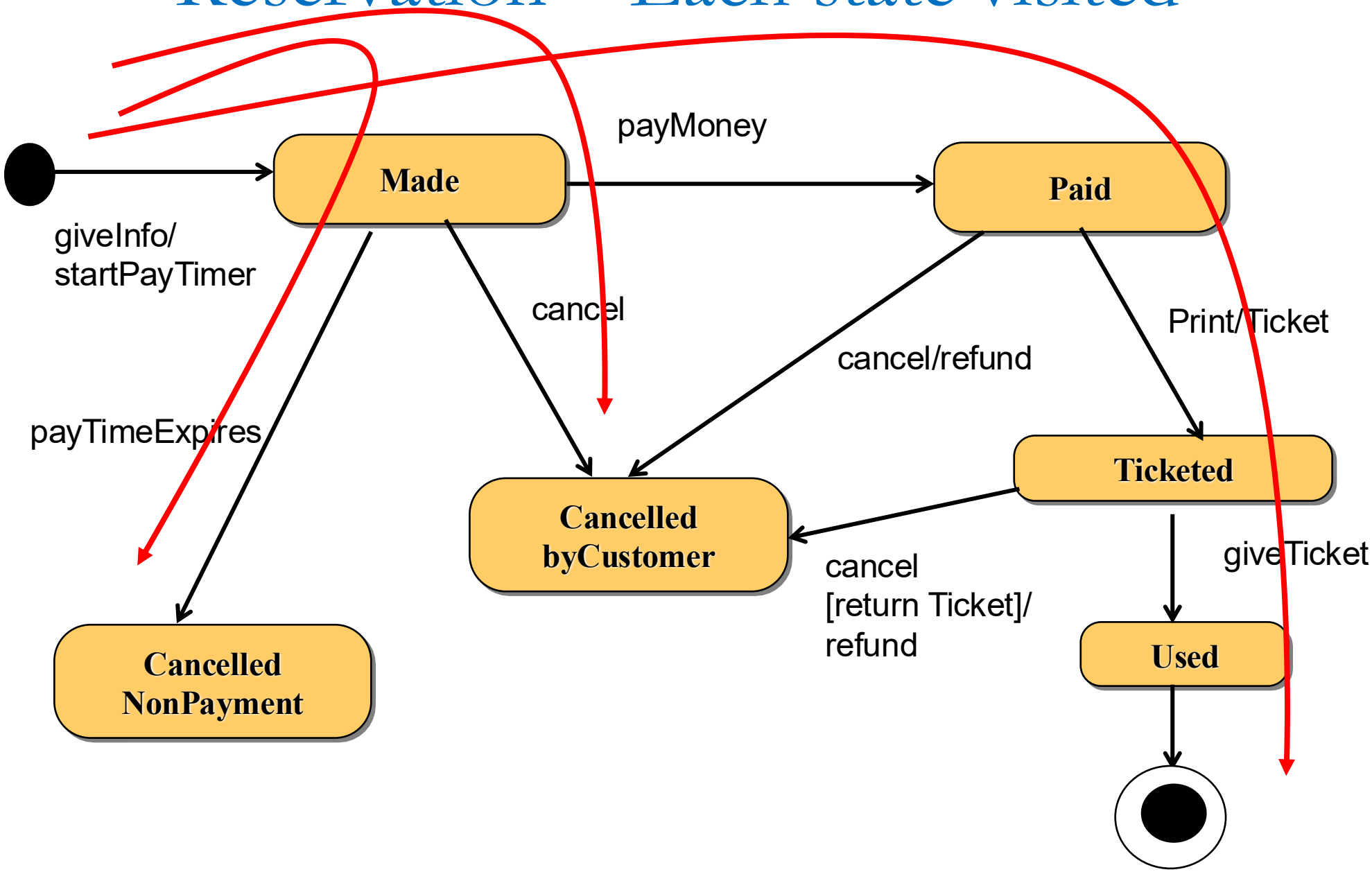
- Consists of four columns – current state, event, action, and next state

Current State	Event	Action	Next State
Start	giveInfo	startPayTimer	Made
Made	payMoney	---	Paid

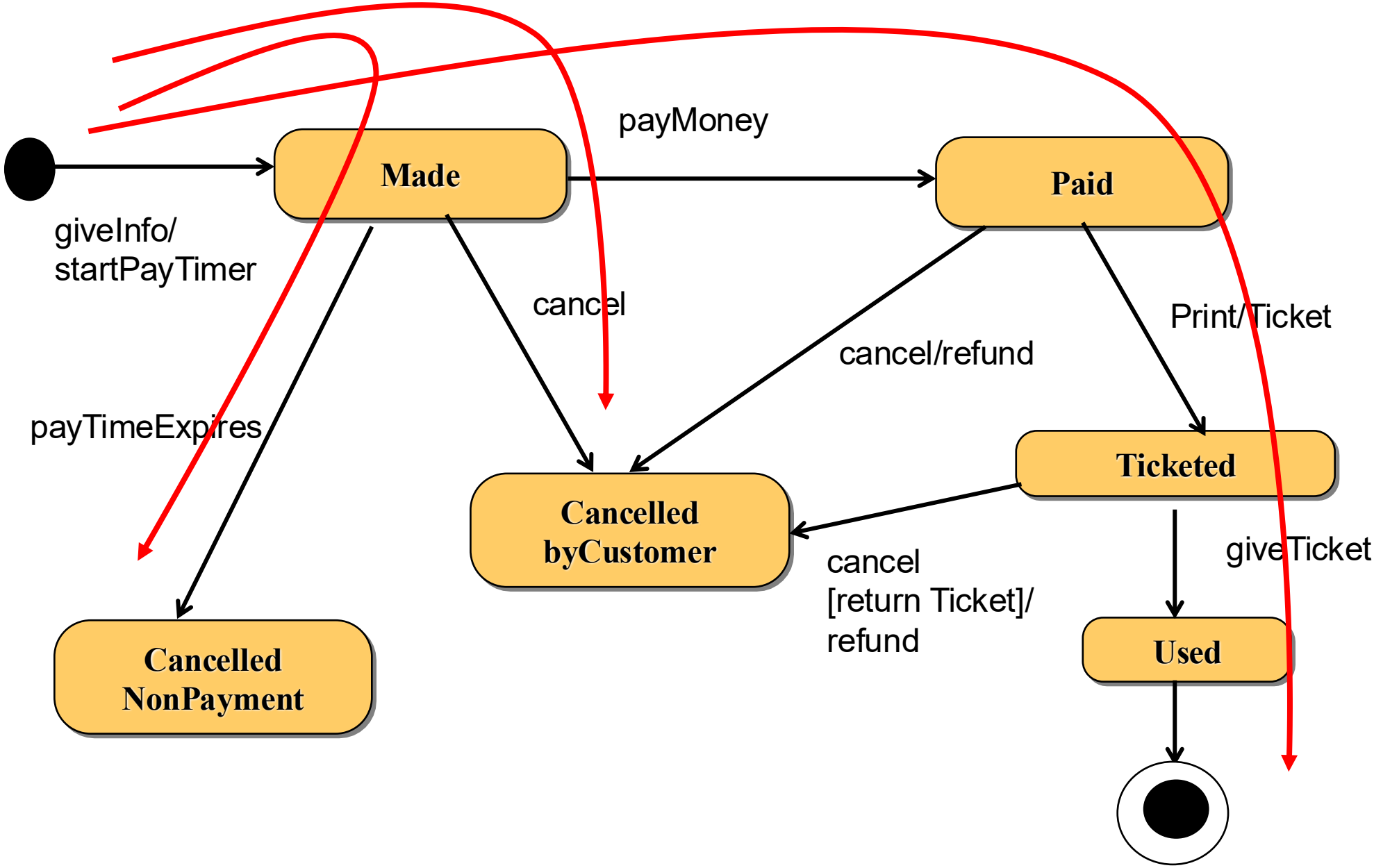
Creating test cases

- Four levels of coverage can be defined
  - Create a set of test cases such that "all states" are **visited** at least once under test
  - Create a set of test cases such that all events are **triggered** at least once under test
  - Create a set of test cases such that "all paths" are **executed** at least once under test
  - Create a set of test cases such that "all transitions" are **exercised** at least once under test

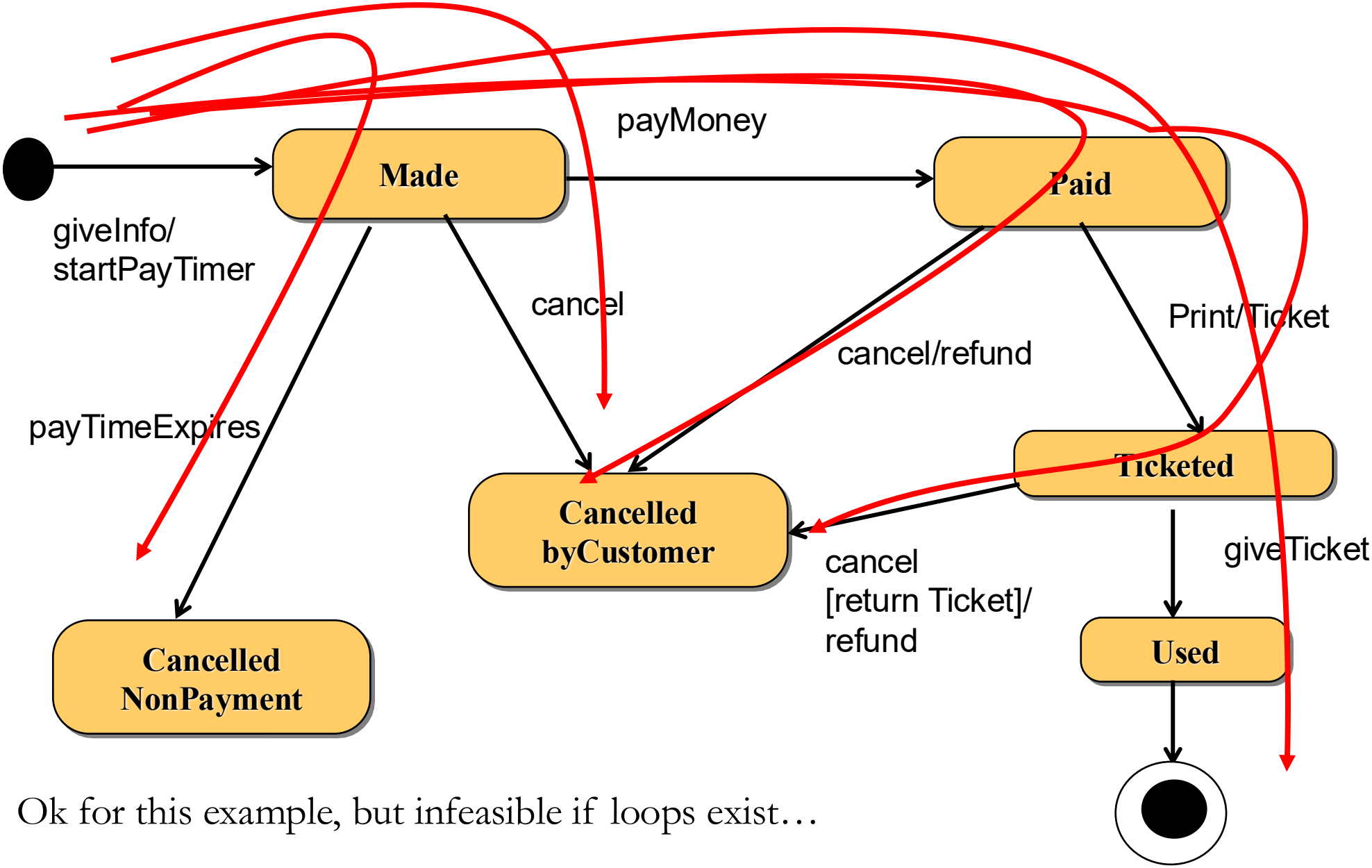
Reservation – Each state visited



Reservation – each event visited

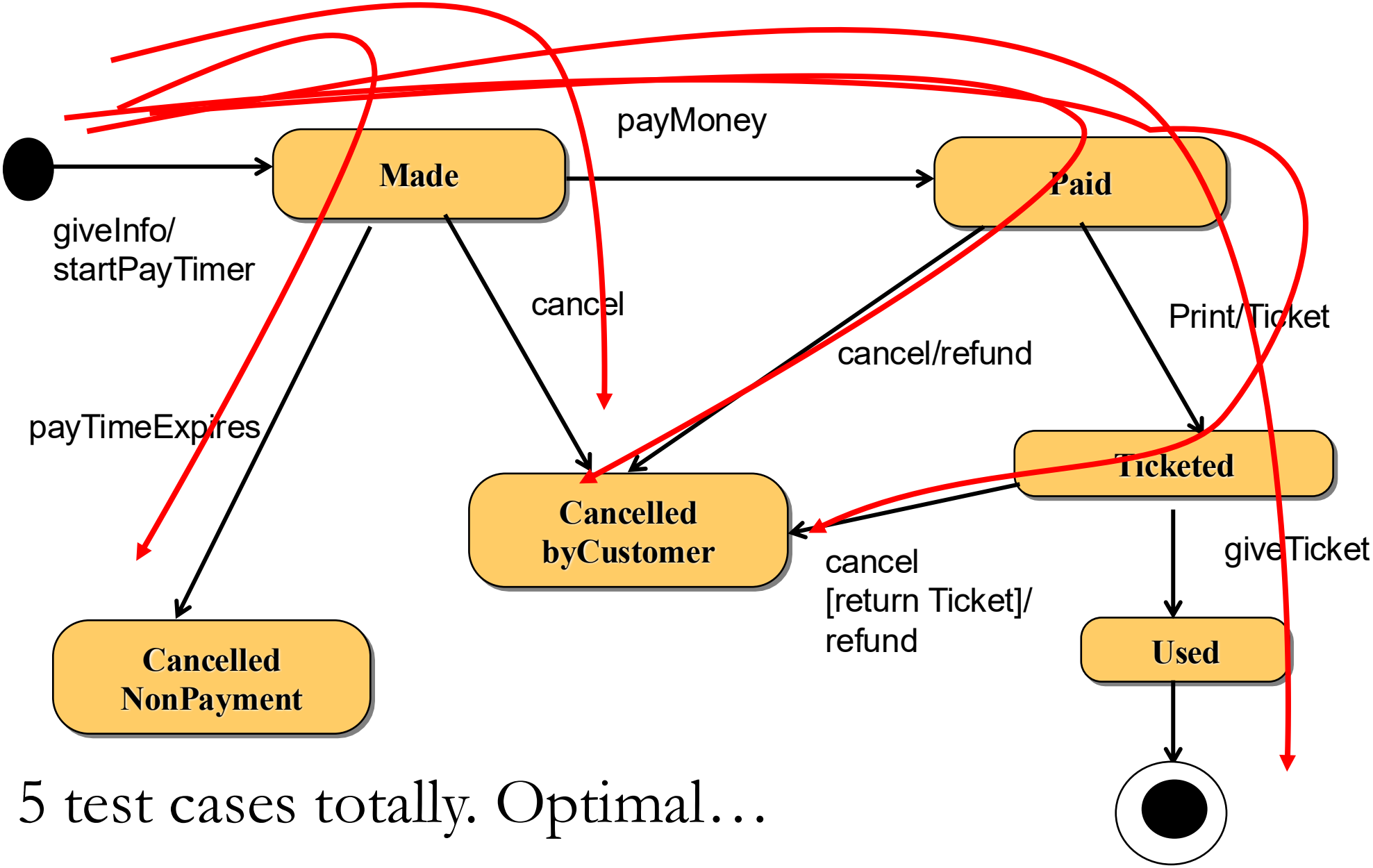


Reservation – each path visited



Ok for this example, but infeasible if loops exist…

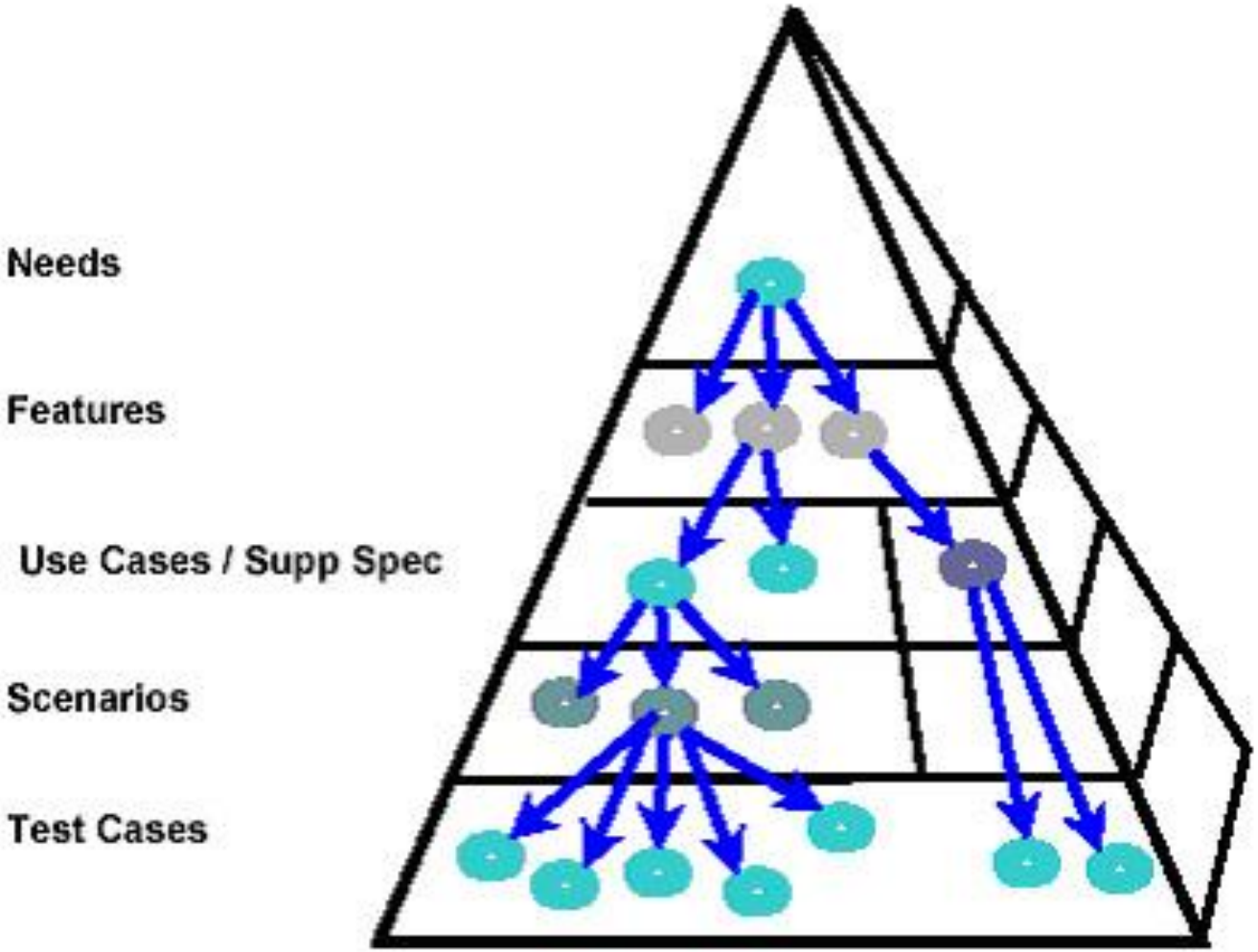
Reservation – each transition exercised



5 test cases totally. Optimal…

Scenarios and Test cases

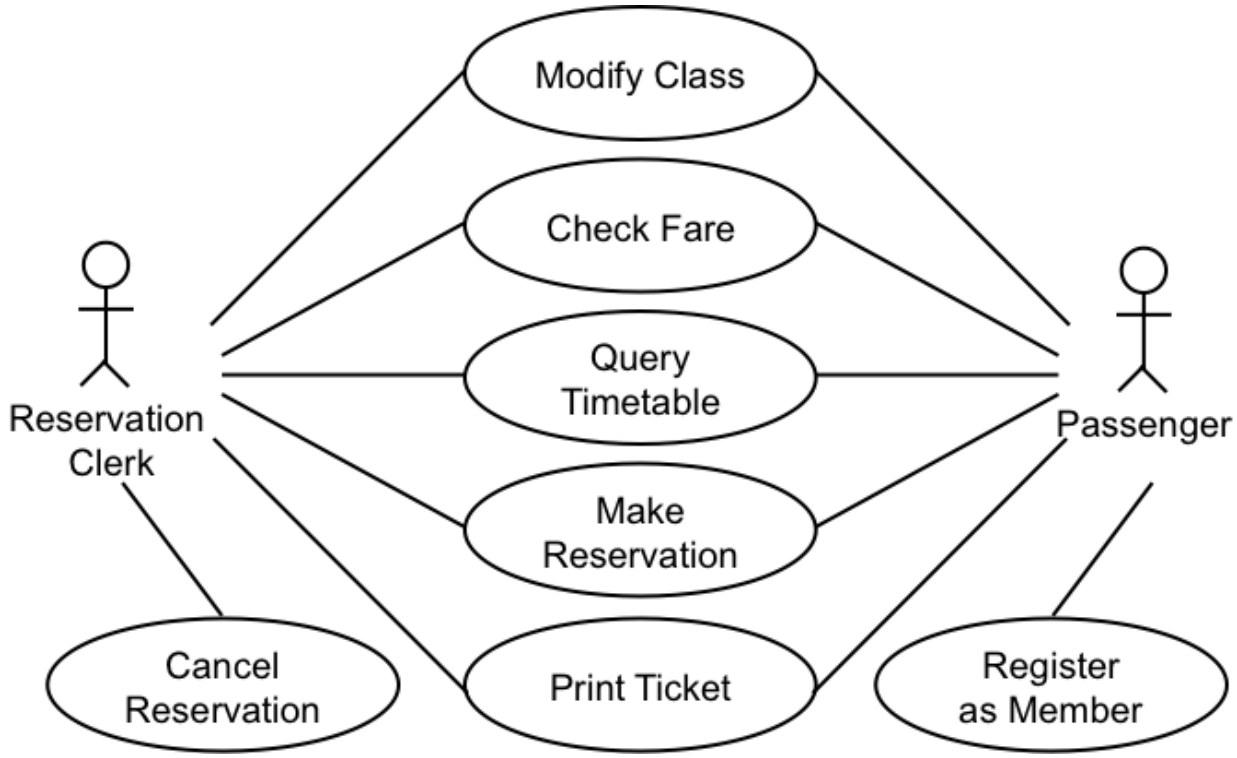
Getting test cases



Identifying Scenarios

- Use case has one basic flow (B) and 0..* alternate flows (A1..An)
- Notation:
  - Basic flow: B
  - Alternative flows: A1, A2, A3, ...
  - Steps in a basic flow: B1, B2, B3, ...
  - Steps in alternative flow 1: A1.1, A1.2, A1.3, ...
  - Steps in alternative flow 2: A2.1, A2.2, A2.3, ...
- Scenario - A possible path through basic and alternate flows that an actor can take.
- Can be Usage, growth or exploratory.
  - Usage from use cases, mostly for functional testing
  - Growth and exploratory for system level testing

Writing test cases



Assume the simplified use case from IRCTC.co.in.

Make Reservation Basic Flow

Use Case	Make Reservation
Actors	Passenger, Reservation Clerk
Purpose	Reserve a seat or berth
Pre-condition	Authorized user is logged in
Overview	Allows the user to make a reservation for a journey.
Normal Flow	1. User logs in 2. User specifies the train and journey details. 3. User specifies passenger details 4. User specifies payment details 5. User selects proceed for transaction
Post Condition	1.User is taken to the payment gateway. 2. Use case make payment initiates
Non-functional requirements	Performance, scalability

Alternate Flows

- A1 No matching trains found
- A2 Source Destination require modification
- A3 Session Expired
- A4 Continue booking another ticket
- A5 Leave in between
- A6 Booking not permitted right now (time, date constraints due to business rules)
- A7 Booking senior citizen ticket
- A8 Changing boarding point
- A9 Check availability
- A10 Close transaction in between
- …Many more

Identifying Scenarios from use cases

Every basic

+

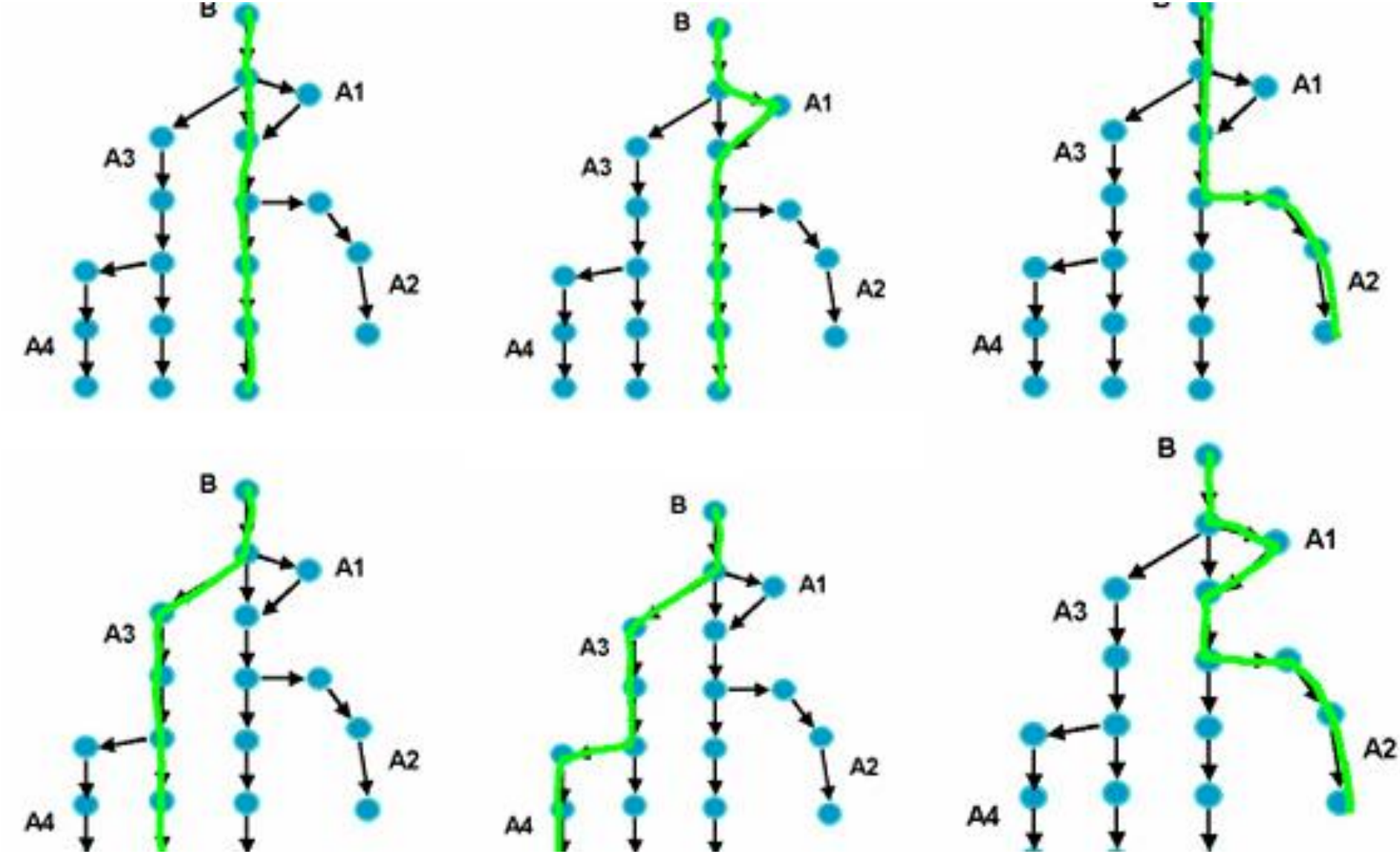
alternate flows individually

+

combinations of alternate flows

- Scenario 1: Basic
- Scenario 2: A1
- Scenario 3: A2
- Scenario 4: A3
- Scenario 5: A4
- Scenario 6: A5
- Scenario 7 A6
- Scenario 8 A7
- Scenario 9 A8
- Scenario 10: A1, A2
- Scenario 11: A3, A4
- Scenario: 12 A4, A5
- Scenario 13: A3, A5
- Scenario 14 A6, A7
- Scenario 15 A7, A8

Possible Scenarios



Regression Testing

Regression

- Loss or deterioration of functionality
- Causes:
  - Introduction of new features creates an undesired interaction with old features
  - Repair of one defect introduces a new defect

When is Regression Testing Done?

- After each new version of a product is created
  - for scheduled new versions
  - for maintenance versions

- Often late in the lifecycle
  - need to pass other tests first
  - last step before delivery

Problems

- Update: maintenance of test suite as product changes
  - Some tests become obsolete
  - Some tests need to be edited
  - Overall suite size grows over time
- Selection: determining which tests to run after a change

Non-functional Testing

Testing system characteristics not related to specific functionality but to *how the system performs* under various conditions

Common Non-functional qualities:

- **Performance** — response time, throughput, latency.
- **Scalability** — ability to handle increasing loads.
- **Security** — protection against unauthorized access.
- **Usability** — user interface intuitiveness and accessibility.
- **Reliability/Availability** — uptime, mean time between failures.
- **Maintainability** — ease of debugging, code clarity.

Performance Testing

Objective: Measure responsiveness, stability, and scalability under workload.

Types of Performance Tests:

- **Load Testing:** Normal workload to measure baseline.
- **Stress Testing:** Beyond normal capacity to find breaking points.
- **Endurance (Soak) Testing:** Long-duration stability.
- **Spike Testing:** Sudden load changes.

Tools: Apache JMeter, LoadRunner, Locust, Gatling, etc

Key Metrics: Response time, throughput, resource utilization (CPU/memory).

Security and Usability Testing

Security Testing:

- Identify vulnerabilities (OWASP Top 10).
- Techniques: Penetration testing, fuzz testing, static/dynamic code analysis.
- Tools: Burp Suite, OWASP ZAP, SonarQube Security, Snyk.
- Metrics: Number of vulnerabilities, severity distribution.

Usability Testing:

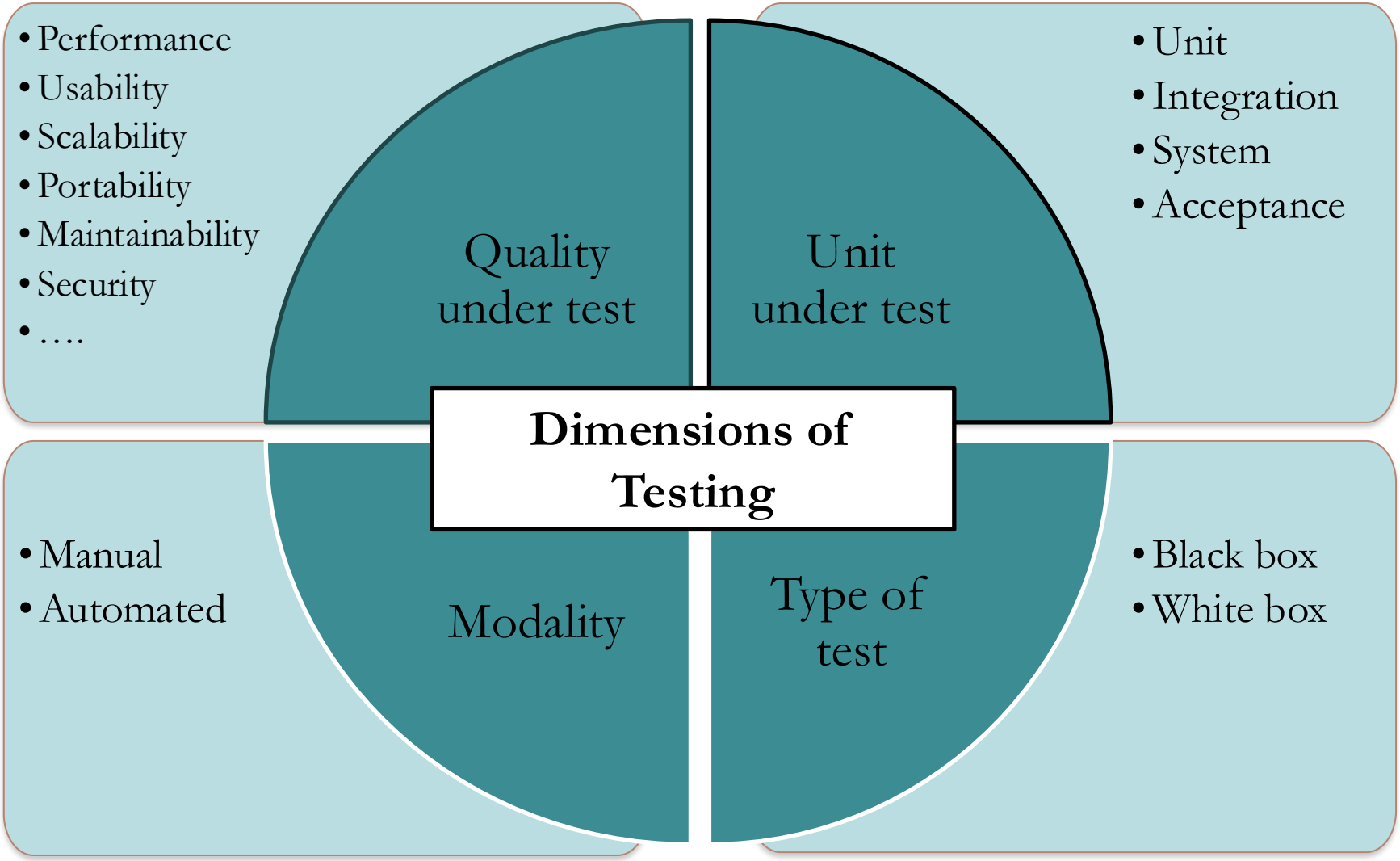
- Evaluate user experience, accessibility, and interaction ease.
- Methods: Task completion rate, user satisfaction surveys, eye-tracking.
- Standards: ISO 9241-210 (Human-Centered Design).

Summary

- Testing is integrated throughout SDLC — not an afterthought.
- Static techniques (inspections, reviews) prevent defects before execution.
- Dynamic techniques (unit, integration, system, acceptance) detect defects through execution.
- Functional and non-functional testing ensure both correctness and quality attributes.
- Structured test design (EP, BVA, Decision Tables, State Transition) maximizes efficiency and coverage.

TESTING – MODULE 2

Various Dimensions of Testing



Writing Test Cases and Test Plans

- Identify the test cases and implement them
  - A *test case* is an explicit set of instructions designed to detect a particular class of defect in a software system.
    - A test case can give rise to many tests (test case instances).
    - Each test is a particular running of the test case on a particular version of the system.
  - The set of test cases constitutes a "test suite"
  - May categorize into critical, important, optional tests (level 1, 2, 3)

- Identify a subset of the tests as regression tests

Test plans

A *test plan* is a document that contains a complete set of test cases for a system

- The test plan is one of the standard forms of documentation.
- If a project does not have a test plan:
  - Testing will inevitably be done in an ad-hoc manner.
  - Leading to poor quality software.
- The test plan should be written long before the testing starts.
- You can start to develop the test plan once you have developed the requirements.

Information to include in a test case

A. Identification and classification:

- Each test case should have a number, and may also be given a descriptive title.
- The system, subsystem or module being tested should also be clearly indicated.
- The importance of the test case should be indicated.

B. Instructions:

- Tell the tester exactly what to do.
- The tester should not normally have to refer to any documentation in order to execute the instructions.

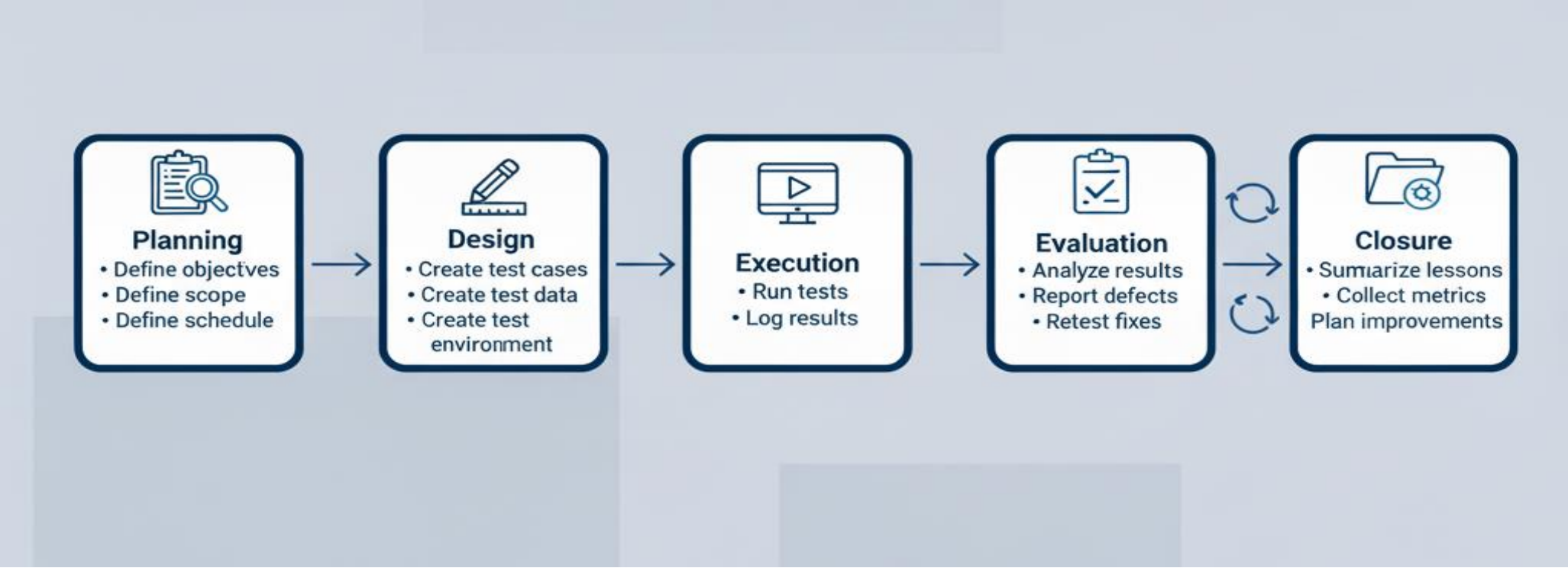
C. Expected result:

- Tells the tester what the system should do in response to the instructions.
- The tester reports a failure if the expected result is not encountered.

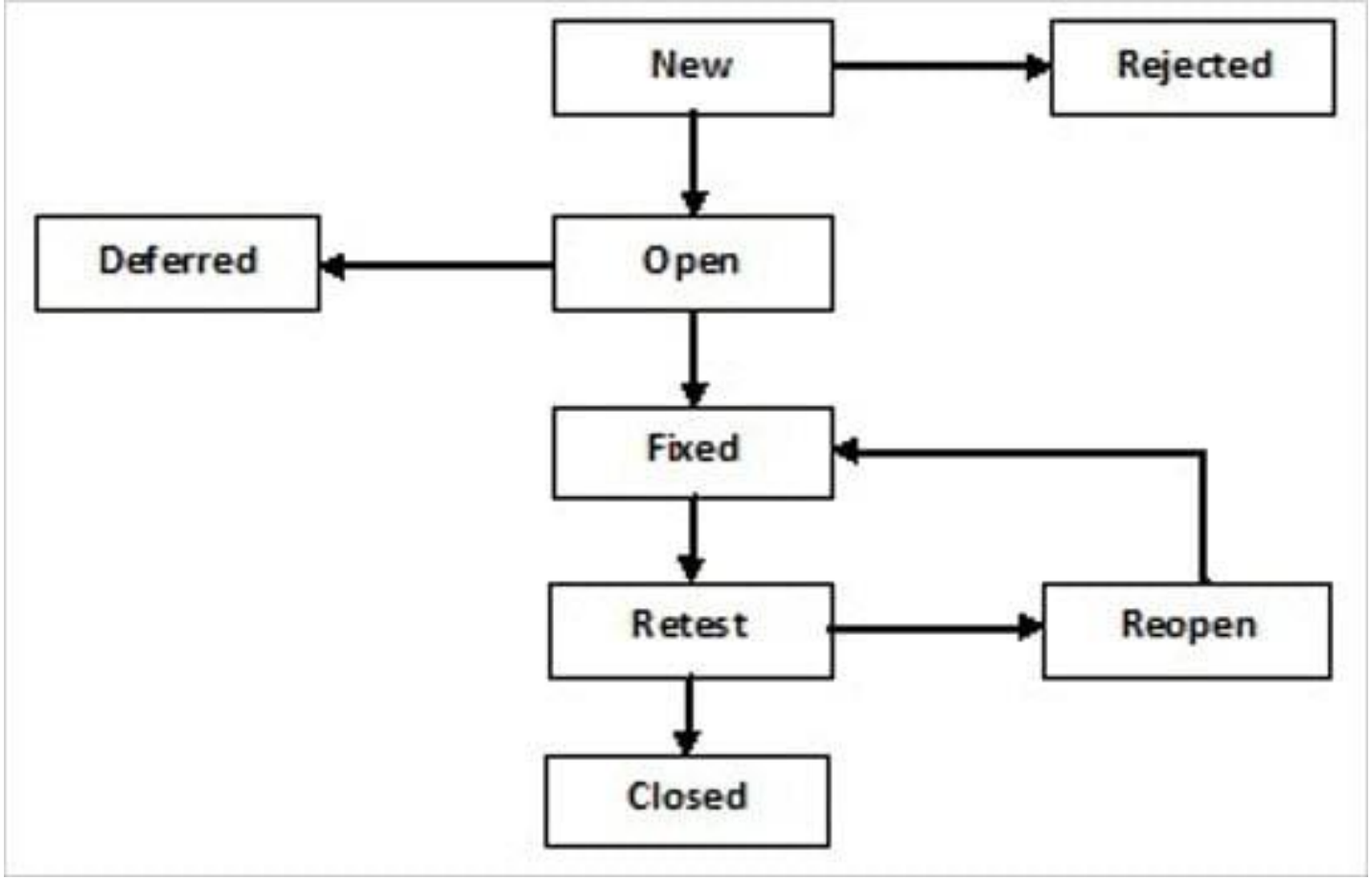
D. Cleanup (when needed):

- Tells the tester how to make the system go 'back to normal' or shut down after the test.

Testing process



Defect Lifecycle

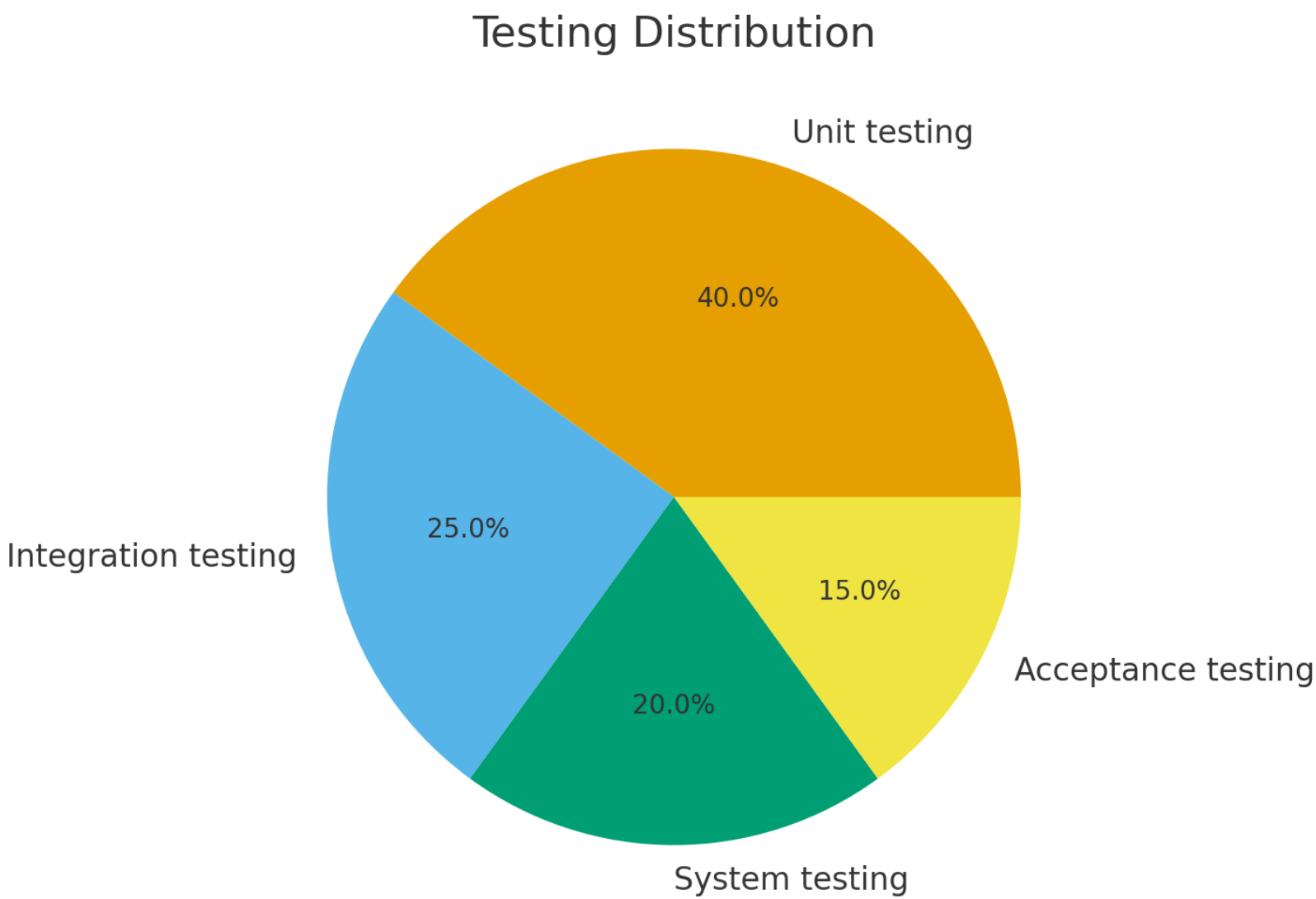


Severity vs Priority:

- Severity: Impact on system.
- Priority: Urgency for fixing.

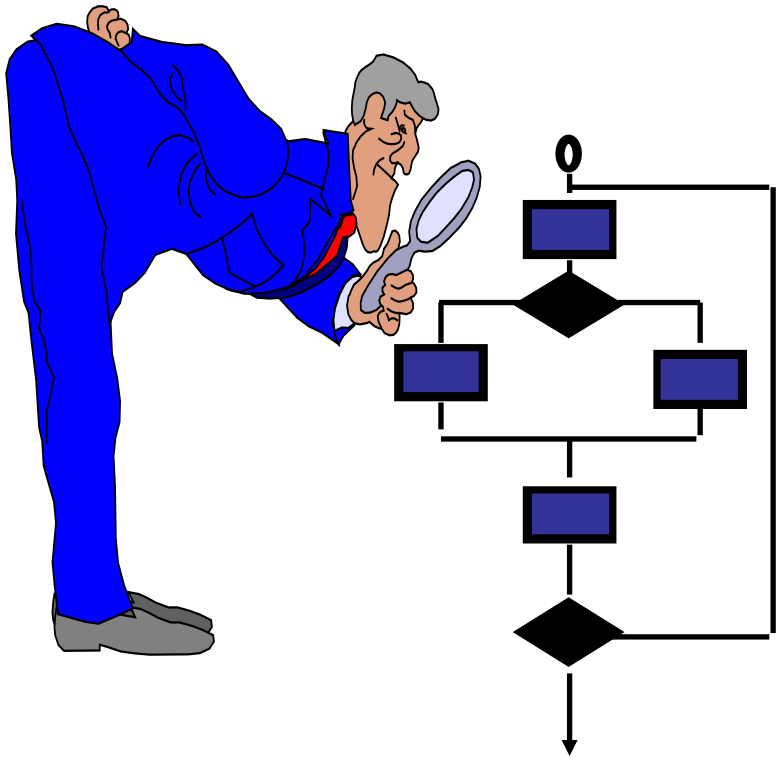
Common Classifications: Blocker, Critical, Major, Minor, Trivial

Designing effective test suites



Next up: White Box Testing !

White-Box Testing



... our goal is to ensure that all statements and conditions have been executed at least once …

White-box testing

- Also called ‘glass-box’ or ‘structural’ testing

- Testers have access to the system design
  - They can
    - Examine the design documents
    - View the code
    - Observe at run time the steps taken by algorithms and their internal data

  - Individual programmers often informally employ glass-box testing to verify their own code

White Box Methods

Can be applied at all levels of system development – unit, integration, and system

- Coverage (Control flow)
  - Statement
  - Branch
  - Condition
- Dataflow
- Mutation

Statement Coverage

- Execute each statement in the program
- Considered minimum criterion for most unit testing
- May be difficult to achieve for error cases

Example Program

```
1:  if (a < 0) {
2:      return 0 }
3:  r = 0;
4:  c = a;
5:  while (c > 0) {
6:      r = r + b;
7:      c = c - 1; }
8:  return r;
```

Statement tests

a = 3, b = 4

executes 1, 3, 4, 5, 6, 7, 5, 6, 7, 5, 6, 7, 5, 8

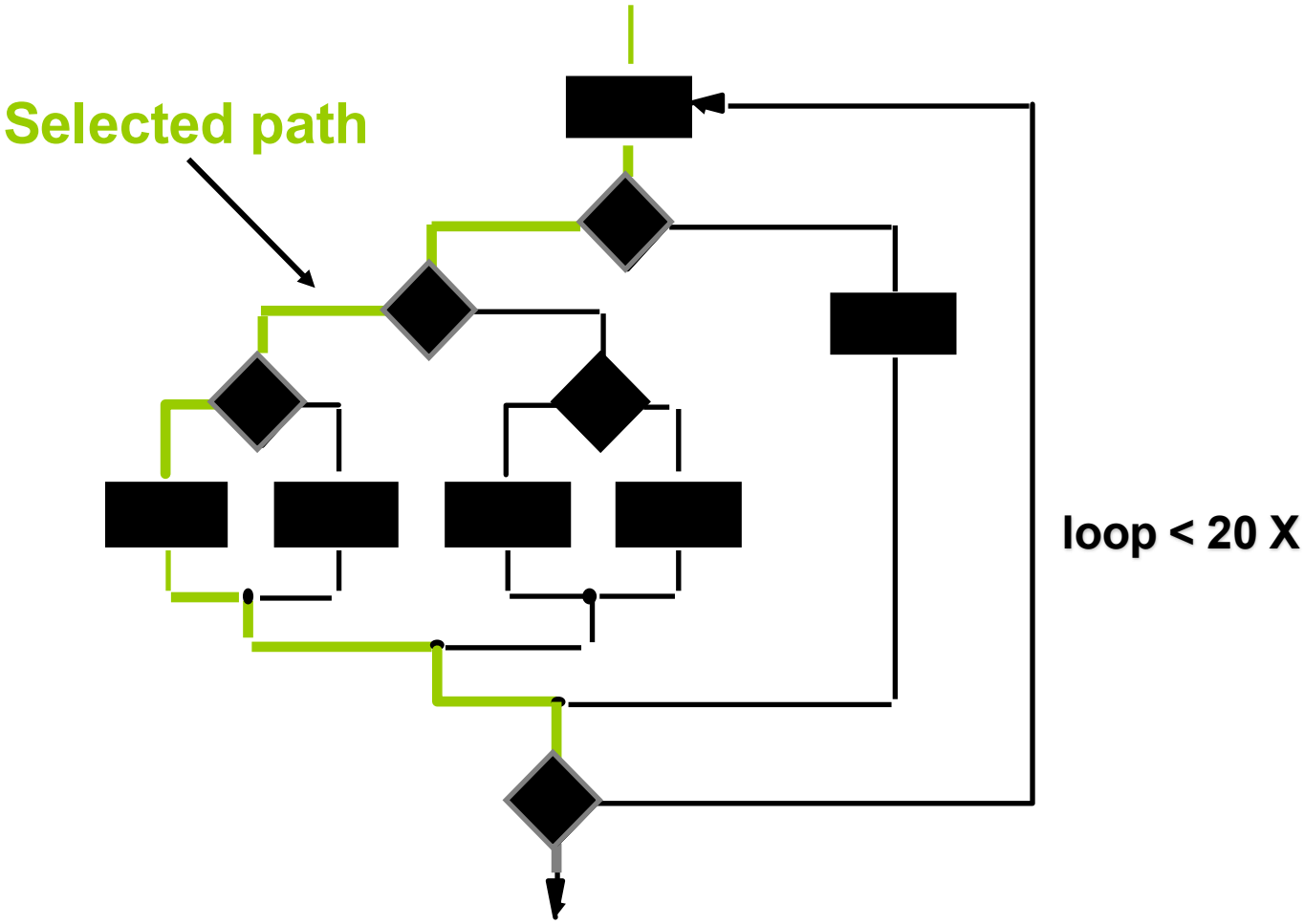
a = -3, b = 2

executes 1, 2

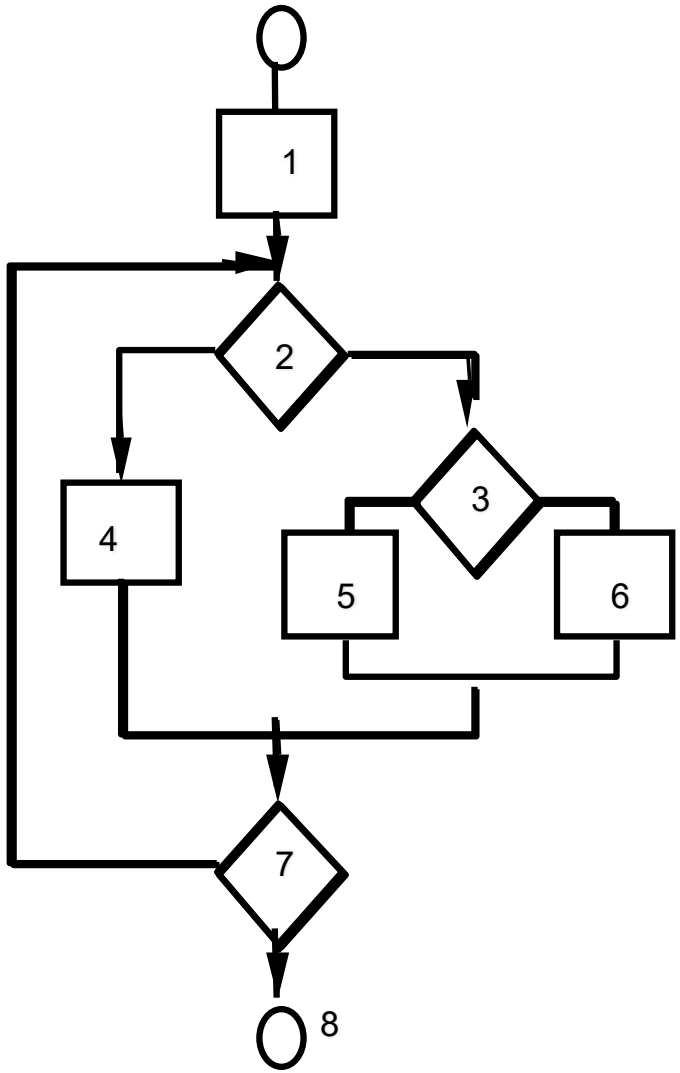
Branch Coverage

- Execute each branch of the program at least once
- Differs from statement coverage only for "if" statements without "else"s and case statements without default cases.

Structured Testing



Basis Path Testing



Next, we derive the independent paths:

V(G) = 4 (there are four paths)

Path 1: 1,2,3,6,7,8

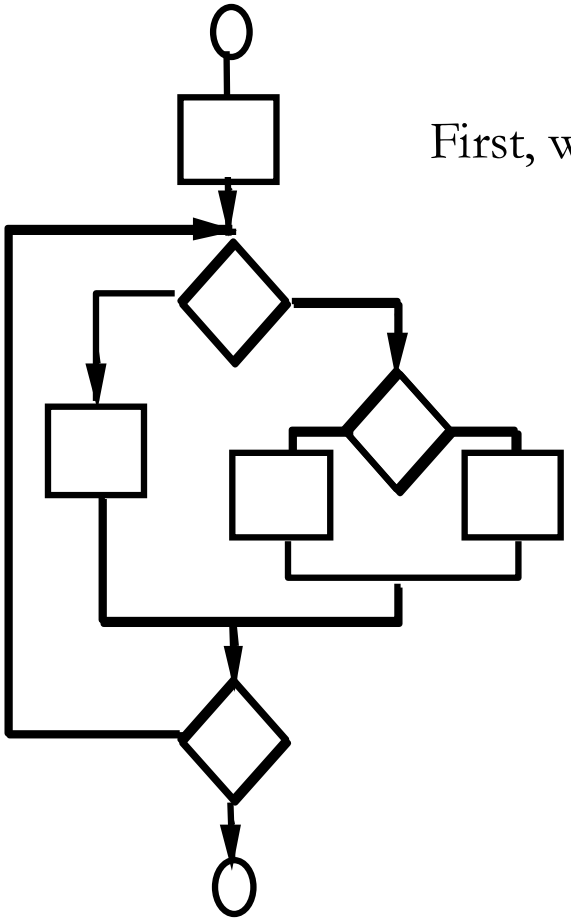
Path 2: 1,2,3,5,7,8

Path 3: 1,2,4,7,8

Path 4: 1,2,4,7,2..,8

Finally, we derive test cases to exercise these paths

Basis Path Testing



First, we compute the **McCabe's cyclomatic complexity:**

number of simple decisions + 1

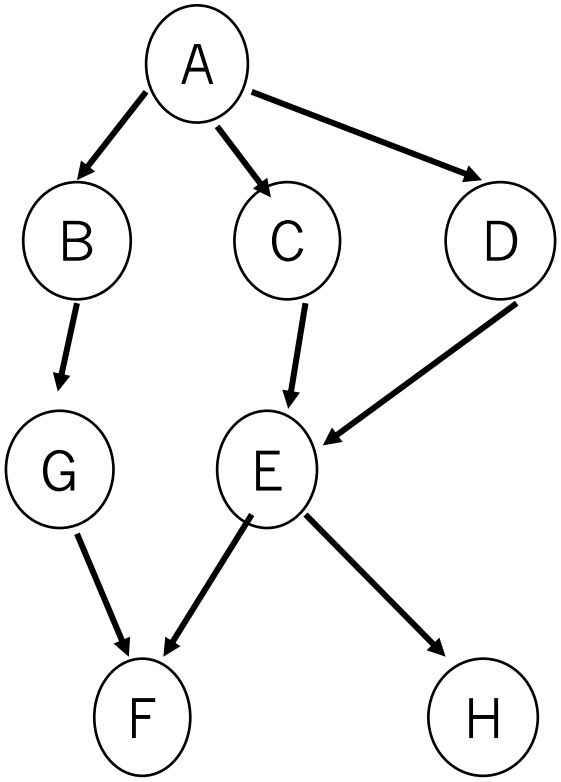
or

number of enclosed areas + 1

In this case, V(G) = 4

If decisions are not binary, this can't be used for computation of cyclomatic complexity

Cyclomatic complexity



Cyclomatic complexity of a graph is

C = edges – nodes + 2

Moving from
Control flow to Data Flow…

Why Data flow testing?

Look at this simple C program

```
#include <stdio.h>
main() {
    int x;
    printf ("%d",x);
}
```

What value for x will be printed ?

Dataflow Testing

Definitions and Uses

- Defining node
  - input statement
  - lhs of assignment
- Usage node
  - output statement
  - rhs of assignment
  - control statement

- **DU testing**: execute each du-path of each variable

Example Program

```
1:  r = 0;
2:  c = a;
3:  while (c > 0) {
4:      r = r + b;
5:      c = c - 1; }
6:  return r;
```

Def (c) = {2, 5}
Def (r) = {1, 4}

Use (c) = {3, 5}
Use (r) = {4, 6}

Example DU Paths

Def (c) = {4, 7} Use (c) = {5, 7}

Def (r) = {3, 6} Use (r) = {6, 8}

du-paths for c:

4 - 5, 4 - 5 - 6 - 7, 7 - 5, 7 - 5 - 6 - 7

du-paths for r:

3 - 4 - 5 - 6, 3 - 4 - 5 - 8,

6 - 7 - 5 - 6, 6 - 7 - 5 - 8

Test Cases for DU Paths

a = 2

1 - 3 - 4 - 5 - 6 - 7 - 5 - 6 - 7 - 5 - 8

Covers du-paths:

4 - 5, 4 - 5 - 6 - 7, 7 - 5, 7 - 5 - 6 - 7

3 - 4 - 5 - 6, 6 - 7 - 5 - 6, 6 - 7 - 5 - 8

Metrics and Reporting

- **Test Metrics:** Coverage, defect density, DRE (Defect Removal Efficiency), flakiness rate.

- **Process Metrics:** Cycle time, MTTR (Mean Time to Repair), MTBF (Mean Time Between Failures).

Dashboards: Jenkins, Allure, TestRail.

Other Testing techniques

Other White box testing techniques

- Loop Boundary Analysis
- Data Dependency Testing
- Path Predicate Testing
- Combinatorial Testing
- Error Guessing
- Mutation Testing
- Model Based Testing
- Static Program Analysis

Loop Boundary Analysis

Testing the boundary conditions within loops

```
def calculate_average(numbers):
    total = 0
    count = 0
    for num in numbers:
        total += num
        count += 1
    return total / count
```

Test cases:

- Test Case 1: Empty input array
- Test Case 2: Array with a single element
- Test Case 3: Array with multiple elements

Data Dependency Testing

Focuses on identifying and testing relationships between different variables and data elements

Example: Testing a function that calculates the discounted price based on a product's original price and discount percentage.

```
def calculate_discounted_price(original_price, discount_percentage):
        discounted_price = original_price - (original_price * discount_percentage / 100)
 return discounted_price
```

Test cases:

- Test Case 1: Positive original price and discount percentage
- Test Case 2: Zero original price with non-zero discount percentage
- Test Case 3: Negative discount percentage

Path Predicate Testing

Analyzing the predicates (Boolean expressions) within the code to create test cases that cover specific paths based on predicate outcomes

Example: Testing a function that determines the category of a product based on its price.

```
def determine_category (price):
    if price < 0:
        return "Invalid"
    elif price < 100:
        return "Low"
    elif price < 1000:
        return "Medium"
    else:
        return "High"
```

Test cases:

- Test Case 1: Price is negative
- Test Case 2: Price is between 0 and 99
- Test Case 3: Price is between 100 and 999
- Test Case 4: Price is 1000 or greater

Combinatorial Testing

Also known as pairwise testing, involves a subset of test cases that cover all possible combinations of input parameters.

Example: Testing a function that validates user login credentials.

```
def validate_login(username, password):
    # Validation logic
    pass
```

Test cases:

- Test Case 1: Valid username and password
- Test Case 2: Valid username with incorrect password
- Test Case 3: Invalid username with valid password
- Test Case 4: Invalid username and password

Error Guessing

Test cases are designed based on common mistakes or areas of concern that specific coverage criteria

Example: Testing a function that sorts a list of numbers.

```
def bubble_sort (numbers):
    # Sorting logic
```

Test cases:

- Test Case 1: Empty list
- Test Case 2: Already sorted list
- Test Case 3: List with duplicate numbers

Mutation Testing

Making small deliberate changes (mutations) to assess effectiveness of test suite in detecting the changes

Example: Testing a function that calculates the factorial of a number.

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Mutation:

- Change ‘result *= i’ to ‘result += i’
- Test Case: ‘factorial (5)’ should fail due to incorrect result.

How many test cases do we need?

- A web site must operate correctly with different browsers – Internet Explorer 6.0, 6.5, and 7.0, Netscape 6.0, 6.1, and 7.0, Mozilla 3.0, and Opera 7; using different plug-ins – RealPlayer, MediaPlayer, or none; running on different client operating systems – Windows 95, 98, ME, NT, 2000 and XP; receiving pages from different servers – IIS, Apache, and WebLogic; running on different server operating systems – Windows NT, 200, and Linux

Pairwise testing

- If a system had four different input parameters and each one could take on one of three different values.
  - Number of combinations – 3^4
  - Pairwise input combinations – 9
- If a system had thirteen different input parameters and each one could take on one of three different values.
  - Number of combinations – 3^{13}
  - Pairwise input combinations – 15
- If a system had twenty different input parameters and each one could take on one of ten different values.
  - Number of combinations – 10^{20}
  - Pairwise input combinations - 180

Roles of People in testing…

Who contributes to Software Quality?

- Testing involves multiple stakeholders with distinct responsibilities.
- Roles vary by organization size, maturity, and development model (Waterfall, Agile, DevOps).
- Core groups: **Testers, QA Engineers, Test Leads/Managers, Developers, Product Owners, Automation Engineers**.

Examples:

- In Agile teams, testing roles are shared across developers and testers.
- In regulated domains (medical, aerospace), formal QA roles are mandatory.

Roles of people in Software Testing

- The first pass of unit and integration testing is called *developer testing*.
  - Preliminary testing performed by the software developers who do the design.

- *Independent testing* may be performed by separate group.
  - They do not have a vested interest in seeing as many test cases pass as possible.
  - They develop specific expertise in how to do good testing, and how to use testing tools.

Testing performed by users and clients

– *Alpha testing*
  - Performed by the user or client, but under the supervision of the software development team.

– *Beta testing*
  - Performed by the user or client in a normal work environment.
  - Recruited from the potential user population.
  - An *open beta release* is the release of low-quality software to the general population.

– *Acceptance testing*
  - Performed by users and customers.
  - However, the customers do it on their own initiative.

Summary

- **Test Management:** Clear planning, documentation, and defect lifecycle tracking ensure organized and repeatable testing.
- **Structural Testing:** White-box techniques and code coverage illuminate gaps not visible from black-box tests.
- **CI/CD & DevOps:** Testing shifts left and right, becoming continuous and integrated across the delivery pipeline.
- **Advanced Techniques:** Mutation testing, model-based testing, Pairwise testing, etc., extend detection and automation capabilities.
- **Metrics:** Coverage, MTTR, and quality dashboards support informed engineering decisions.
- **Shared Quality Ownership:** Developers, testers, automation engineers, product teams, and DevOps all contribute to quality.